Faculty of Engineering Sciences

University of Heidelberg

Study Project in Computer Engineering

submitted by Enrica Schmidt

17 August 2026

# Reconfigurable Custom Instruction Set Extensions for RISC-V

This Study Project has been carried out by Enrica Schmidt

at the Institut für Technische Informatik

under the supervision of Prof. Dr. Dirk Koch

## Eigenständigkeitserklärung

Hiermit versichere ich, dass ich die Prüfungsleistung "Reconfigurable Custom Instruction Set Extensions for RISC-V"

1. selbstständig angefertigt habe und
2. keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe.
3. Sämtliche wörtliche oder sinngemäß übernommenen Textstellen habe ich als solche kenntlich gemacht.

---

**Ort, Datum, Name**

## Angaben zu verwendeten KI-basierten elektronischen Hilfsmitteln

Zur Dokumentation der verwendeten Hilfsmittel ist der schriftlichen Ausarbeitung ein besonderer Anhang hinzugefügt, der eine Liste und Beschreibung aller verwendeten KI-basierten Hilfsmittel enthält. Der besondere Anhang zur Dokumentation der verwendeten Hilfsmittel erfüllt folgende Kriterien:

1. Auflistung der Ziele, für die die KI-basierten Hilfsmittel in der vorliegenden Arbeit eingesetzt wurden,
2. Dokumentation der Verwendungsweise der KI-basierten Hilfsmittel,
3. Nennung der Kapitel und Abschnitte der vorliegenden Arbeit, in denen die KI-basierten Hilfsmittel eingesetzt wurden, um Inhalte zu erzeugen.

Der Gebrauch dieser Hilfsmittel inklusive Art, Ziel und Umfang des Gebrauchs wurde mit meinem Erstbetreuer Prof. Dirk Koch abgesprochen.
Mir ist bewusst, dass insbesondere der Versuch einer nicht dokumentierten Nutzung KI-basierter Hilfsmittel als Täuschungsversuch entsprechend § 12 der Prüfungsordnung zu werten ist:
„Versucht die zu prüfende Person das Ergebnis der Prüfungsleistung durch Täuschung oder Benutzung nicht zugelassener Hilfsmittel zu beeinflussen, kann die betreffende Prüfungsleistung mit „nicht ausreichend“ (5,0) bewertet werden.“

---

**Ort, Datum, Name**

# Abstract

The RISC-V ISA supports custom instructions, which can be more specialized than the base instructions. Implementing the same functionality in the base ISA would usually require a sequence of instructions, which can instead be replaced by a single custom instruction call. This way a computation can be accelerated. To provide the hardware support for the custom instructions, an eFPGA fabric can be used. The reconfigurable fabric allows modifying the custom instructions in hardware to match the program currently executed.

The ICESOC is an SoC containing two RISC-V cores, two 1KB SRAMs, and an eFPGA fabric to host custom instructions. It has several shortcomings. Firstly, because of mismatched address signals between the cores and the SRAMs, the cores can only access 64 words per SRAM. This reduces the memory available by three quarters. Secondly, the UART outputs are permanently disabled, therefore no UART output is possible in the ICESOC.

This report consists of three contributions. The first contribution is a documentation of the ICESOC. The second contribution is the implementation of firmware to realize an SRAM paging mechanism. This is needed to deal with the little memory available. It allows the cores to request the instructions and the bistream page by page as they are needed. After being requested, the pages are provided to the cores though the SRAM. The third contribution is the implementation of a CRC demo application. The objective of this is demonstrating the use of the SRAM paging mechanism, and showing that a speedup is possible by using a CRC custom instruction on an eFPGA. This is shown in simulation.

# Contents

# 1 Introduction

Modern instruction set architectures (ISAs) have to keep fulfilling legacy requirements, which adds complexity that might be obsolete [1]. To avoid this, RISC-V consists of a base ISA sufficient to run a full software stack [2]. It can include optional standard extensions or custom extensions. Using those custom extensions, a sequence of base instructions can be replaced with a single specialized custom instruction call. This way, a computation can be accelerated. To show this is one aim of this work. The custom instruction can be supported in hardware by static accelerators or by a reconfigurable eFPGA fabric. Here, the latter approach is chosen for its flexibility.

The ICESOC is an SoC, which hosts two ibex RISC-V cores, two 1KB SRAMs, and an eFPGA fabric. The fabric has three slots and can contain up to three custom instructions, which can be used to accelerate the cores. Two operands are passed directly from the cores' register files to the fabric slots, and three results are returned, one of which is written to the destination register of the instruction. The ICESOC is embedded in a Caravel harness, which contains a management core that can communicate with the other two cores. The eFPGA can be configured in one of three ways. Either via UART, through a bitbang interface, or by one of the cores. Here, the ibex core is used to configure the fabric. For this, the bitstream should be provided to the core through SRAM, just like the core's instructions. The data can be written to SRAM through UART or by the Caravel management core.

The ICESOC has several shortcomings. Firstly, its output enable signals are flipped, resulting in permanently disabled UART outputs. Secondly, there is a mismatch between the signal the core uses to address the SRAM, which is a byte-address, and the address signal expected by the SRAM, which is a word address. This results in the cores only being able to access every fourth SRAM word, effectively shrinking the capacity of both SRAMs to 64 words each. To deal with this limited SRAM size and to bypass the UART connection that works only one way, a paging mechanism is implemented in which the Caravel management core is used to provide data to the ibex core. The ibex core can request its instructions and the bitstream for configuring the eFPGA page-wise. The management core then provides that data.

This report contains three main contributions, which are structured as follows. Section 3 is intended as a short documentation of the ICESOC. In Section 4, the SRAM paging mechanism implemented to address the ICESOCs shortcomings is introduced. Finally, a CRC application is implemented and introduced in Section 5. It aims to demonstrate the SRAM paging in simulation and to measure a possible speedup achievable by using custom instructions on an eFPGA. The implementations were tested in RTL simulation.

# 2 Background

## 2.1 Instruction Set Architectures

The instruction set architecture (ISA) of a processor describes the interface between its hardware and software [3]. This includes the supported machine instructions, the size and number of registers, as well as the memory addressing. The concrete implementation of an ISA is realized by a microarchitecture [3]. Different microarchitectures can implement the same ISA. They can differ in implementaion details, like pipeline and cache organization [3]. There are two main approaches to designing an ISA: Complex Instruction Set Computers (CISC), and Reduced Instruction Set Computers (RISC),which are described below.

A CISC processor uses complex instructions that can do many tasks in one instruction, like a MUL instruction that loads the operands from memory, multiplies them and stores the result back to memory [4]. It is designed for easy writing of assembly. Programs written for CISC are relatively short because few instructions are needed per program since each instruction encapsulates many tasks at once. This means that a program needs little memory to be stored. CISC internally uses microcode to translate its complex instructions into the correct hardware control signals to avoid hardwiring all this complex logic [5]. Because CISC processors have kept adding specialized instructions, backwards compatibility needs to be ensured to be able to run old software [6]. When instructions become outdated, this turns into an obstacle, since they still have to be included to fulfill the legacy requirements. A CISC instruction usually takes more than one clock cycle to execute, so its clock cycles per instruction (CPI) are relatively high. Also different instructions can have different execution times. This makes pipelining the instructions more difficult. Since each instruction encapsulates load and store instructions, operands need to be loaded again for every instruction, even if the previous instruction already loaded them.
The instructions can be of varying length. This and the sheer amount of different instructions makes the instruction decoding more difficult. The hardware for a CISC ISA is usually relatively complex and needs many transistors, which translates to more area, because the instructions are so specialized. CISC tries to reduce the time a program needs to run by reducing the number of instructions per program at the cost of a higher CPI.

$\frac{time}{program} = \frac{time}{clock\_cycle} * \frac{clock\_cycle}{instruction} * \frac{instructions}{program}$ [2].

RISC takes the other approach to reduce its execution time per program by reducing the number of cycles needed per instruction (CPI) at the cost of more instructions per program. RISC aims to reduce the complexity of its ISA by using less instructions and more simple, general ones. This makes pipelining and decoding easier and the hardware needed to execute the instructions is simplified. Most instructions execute in a single clock cycle [2], or a pipeline is used, which aims for a thoughput of one instruction per cycle. Less transistors are needed which reduces area and power consumption. The downside is that more instructions are needed to write a program because every instruction itself does not encapsulate as many tasks as the CISC instructions do. This means the program code will be longer and therefore less memory efficient. Writing the assembly code becomes more tedious because of the simpler instructions and the requirements of manually handling many things like loads and stores, which were included in the complex CISC instructions before. However the instruction decoding is simplified because less instructions have to be distinguished and they all have the same length.

## 2.2 RISC-V with custom instructions

RISC-V is a concrete open-source ISA that follows the RISC design priciple. It describes the instructions that a microarchitecture implementing it needs to be able to execute. RISC-V is designed to be modular and easily extensible, and consists of a small base ISA, sufficient to run a full software stack, as well as optional standard extensions [2]. Defining own custom instructions is also possible. For this, space for custom extensions is reserved in the RISC-V instruction encoding space, which is guaranteed not to conflict with the standard extension opcode space [7]. Since RISC-V is so modular, a processor implementing only its minimal base ISA can be very small and power efficient. This is especially useful for embedded applications [2]. For more complex applications, the ISA can be extended as needed with the standard or custom extensions. Hence RISC-V can adapt to the complexity of the application and does well on systems of different scales.

## 2.3 Accelerating a processor with an eFPGA fabric

Accelerators are hardware, which is designed to perform a specific function, as an extension to a general purpose core [8]. They can achieve higher performance and power efficiency, than using a general purpose processor [8]. Accelerators can be invoked at different levels. One option is instruction-level invocation, where the accelerator is integrated into the processors ISA [9]. In that case, custom instructions could be executed on the accelerator. This is supported by the RISC-V ISA, because it prioritizes modularity [10]. If a certain complex functionality is needed often, an accelerator can be added, which is optimized for this functionality and can execute it faster than if it would be broken up into multiple base RISC-V instructions. So the entire program can be sped up this way. This means that the hardware is very closely tied to the software it runs because the accelerators it uses are specialized for this. But if the program changes and the accelerator is idle most of the time, its cost outweighs the benefits.

To prevent the overhead of unused accelerators, an eFPGA fabric can be used instead of the static accelerators. The implementation of the custom instructions is moved to the fabric, which can be reconfigured even during program execution [1]. This leads to a design where only the permanently needed instructions are implemented in silicon and the custom instructions can be configured to the eFPGA on demand. The fabric can be reconfigured to host the custom instructions needed for a specific program, which adds flexibility. Usually software can be updated easily, but by using custom instructions that need hardware support, this updatability is limited. With reconfigurable hardware, like an eFPGA the possibility to update the software, including the custom instruction used, is reintroduced.

## 2.4 The ICESOC

The ICESOC investigated here is a successor of the FlexBex described in [1]. It combines two Ibex RISC-V cores with an eFPGA fabric, which can host the reconfigurable custom instructions. One of the cores can trigger the reconfiguration of the fabric. Here, the eFPGA fabric is added as an additional execute block into the pipeline of the RISC-V CPU alongside the ALU [1]. The operands for the custom instructions on the fabric come directly from the core's register file, and the result is written back to it directly as well. The operands are the contents of the two source registers specified in the custom instruction, and the returned result is written to the destination register selected with the instruction. Since there are three slots on the eFPGA, the instruction also contains a field to select the result of which slot, and therefore which custom instruction should be picked. This custom instruction format is further described in Section 3.6.1. The eFPGA fabric is tightly integrated into

the RISC-V pipeline. This allows a fine-grained split of the program by executing single instructions on the eFPGA fabric, which can replace a longer sequence of base RISC-V instructions, and therefore provide a speedup. The ICESOC will be described more thoroughly in Section 3.

## 2.5 Related Work

GARP [11] is a hybrid architecture, which uses a MIPS CPU, and an FPGA as a reconfigurable slave computational unit. The FPGA can access the memory directly without processor intervention [11]. As a result it can work independently on the memory, whereas in the ICESOC, it can only execute instructions on the operands passed to it from the register file during the invocation of a custom instruction. In GARP, there is a looser integration of the fabric into the core than in the ICESOC, so it can achieve speedups if the acceleration jobs are relatively large. It requires a coarser grained split of work between the processor and the FPGA.

The Dynamic Instruction Set Computer (DISC) [12] uses an FPGA to host a global controller and different instruction modules, that can be swapped through partial reconfiguration. As opposed to the ICESOC or GARP, the entire processor is implemented on the FPGA, not just a few additional instructions. The global controller is the only part, which is constantly loaded on the FPGA. It provides some default instructions for sequencing, status control, and memory transfer. The reconfiguration possibilites here are more invasive than in the ICESOC or GARP, since the processor is not just extended with a reconfigurable fabric, but it is implemented directly on the fabric, where the instruction modules, which essentially are part of the processor, can be reconfigured.

Greyhound [13] is another SoC with a RISC-V processor and a reconfigurable eFPGA fabric. It works similarly to FlexBex, but unlike FlexBex uses an open-source PDK, and open-source EDA tools [13]. The RISC-V core it uses, the CV32E40X already has a custom instruction interface, which is used [13]. In the ICESOC, the ibex cores have been manually modified to accomodate the interface to the fabric. Just like the ICESOC, Greyhound uses R-type custom instructions with two source registers and one destination register. Greyhound has only one core and one SRAM, unlike ICESOC, which has two of each. Greyhound also has 8 KB built-in SRAM, unlike the ICESOC, which has two times 1KB, of which only a quarter is addressable by the two cores.

# 3 Overview ICESOC

## 3.1 Motivation

The ICESOC was designed for cryptographical applications [14]. It is a multicore SoC, which contains two Ibex RISC-V cores, two 1KB SRAMs, and an eFPGA fabric, which was generated with FABulous. The purpose of the eFPGA is to host custom instructions, which can be used to accelerate the cores. The eFPGA fabric consists of three slots in which the custom instructions can be loaded. They are tightly integrated with the cores and can be reconfigured. The configuration can be triggered from one of the cores.

The custom instructions can be implemented efficiently in hardware and can replace a longer sequence of base RISC-V instructions. Because offloading some custom instructions to the eFPGA can save CPU cycles compared to running the entire program only on a generic RISC-V core, the computation can be accelerated this way. This speedup could also be used to reduce clock speed and therefore power [1].

The ICESOC was designed as part of an open source multi-project wafer (MPW) shuttle sponsored by Google. The SoC uses the SkyWater Open Source PDK and was fabricated in the SKY130 130nm process node. It uses a Caravel harness, which is an SoC that hosts the user project, or mega project (mprj), as well as a management area with a management RISC-V core, an SRAM for the core, and some housekeeping functionality outside of the management area [15]. The user project has a size of 2.92mm x 3.52mm [15], while the entire Caravel harness die has a size of 3.2mm x 5.3mm. In this case the user project is the ICESOC. The layout of the entire chip can be seen in Fig. 1. On the outside, there are the pads, on the bottom are the Caravel internals, like the management core and its SRAM. The user project is on top. On the top left inside the user project is SRAM 1, below is the ibex core, on the top right is SRAM 2 and below that is the flexbex core. The T-shaped eFPGA fabric occupies the center of the user project. The ICESOC can be found at [14].

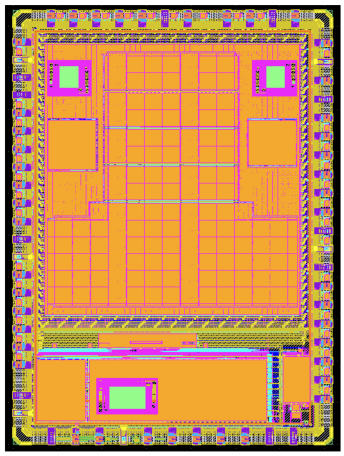

Figure 1: Layout of the Caravel harness with the ICESOC as user project (3.2mm x 5.3mm in total). On the outside are the pads, the bottom contains the Caravel management core and SRAM, on the top is the user project with two RISC-V cores, two SRAMs, and a T-shaped eFPGA fabric in the middle. [16]

## 3.2 Architecture overview

The architecture and hierarchy of the modules within the ICESOC are illustrated in Fig. 2. The ICESOC SoC consists of a Caravel harness with a custom user project. Inside Caravel, there is a management RISC-V core, which is connected to the user project through a wishbone bus. GPIOs from the user project to the pads are available, which can be configured and overwritten by the management core. The user project contains the eFPGA with its configuration interface and the icesoc_top module. The icesoc_top works with a master-slave principle. It contains the two cores as masters, and a UART master. The two SRAMs as well as another UART instance function as slaves. An inter module, which contains an arbiter takes care of orchestrating the communication between the masters and slaves.

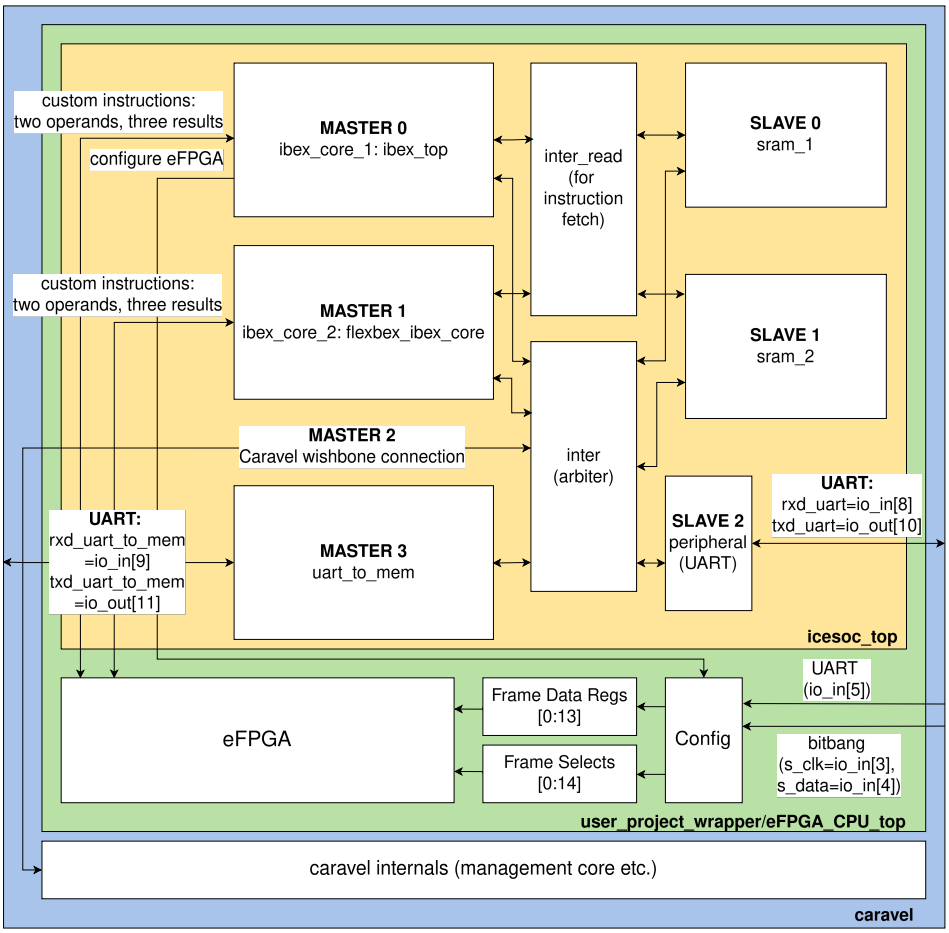


Figure 2: ICESOC hierarchy

## 3.3 The master-slave principle in the ICESOC and arbitration

ICESOC functions with a master-slave principle. There are four masters and three slaves as well as two modules that orchestrate the communication, the inter and inter_read modules. Inter handles the requests by the masters, grants the requests with a priority that is determined by its internal round-robin arbiter, and passes the signals between the master and its requested slave. Inter_read is only responsible for instruction fetching. It therefore only has to coordinate two masters, the two cores, and two slaves, namely the two SRAMs. Because inter_read only handles the instruction fetching, it

only handles read requests from the cores to the SRAMs. This chapter will focus on the functionality of inter, because inter_read is just a simplified version for instruction fetching.

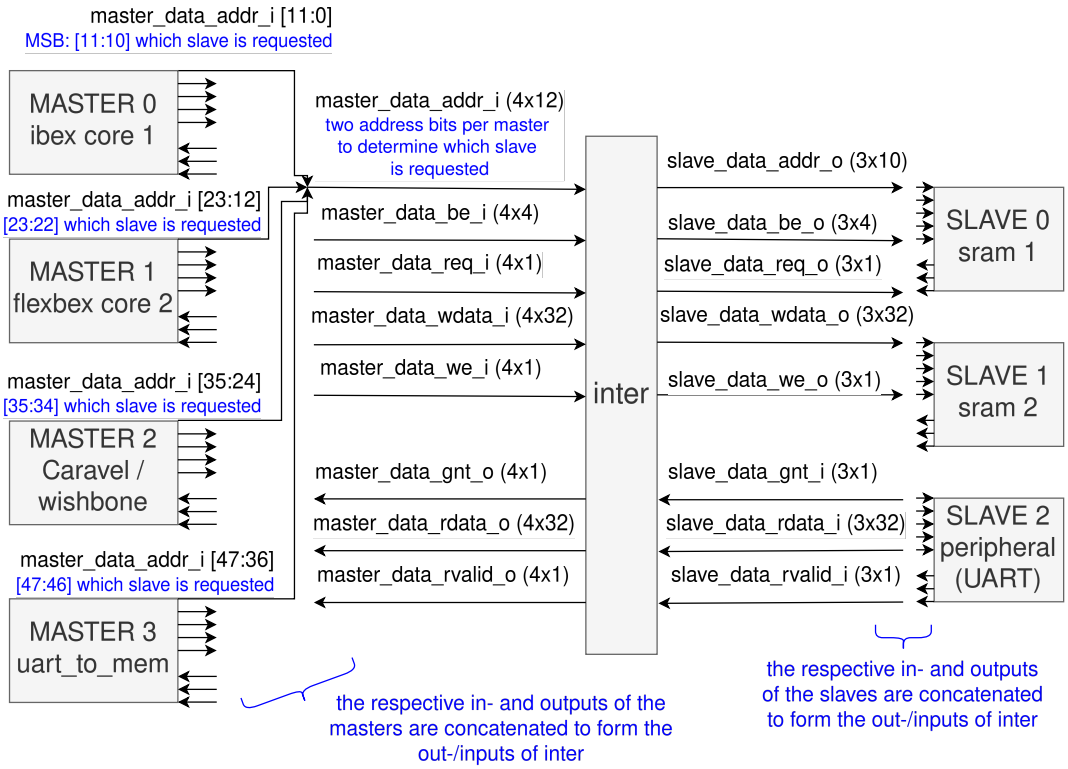


Figure 3: The inter module orchestrates the communication between the four masters and three slaves and arbitrates among the requesting masters.

Each master provides five inputs to the inter module:

- req: 1 bit per master; The request signal
- addr: 12 bits per master; This is the slave address from/to which the master wants to read/write. The two MSbs are used to determine which of the three slaves the master wants to access.
- we: 1 bit per master; write enable signal; 0 if the master wants to read, 1 if it wants to write.
- be: 4 bits per master; byte enable (mask)
- wdata: 32 bits per master; The data to be written to the slave

and receives three outputs from the inter module:

- rdata: 32 bits per master; the data read from the slave
- rvalid: 1 bit per master; rvalid signals that rdata is valid
- gnt: 1 bit per master; grant signal for this master (the request was approved)

The slaves receive the signals from the masters and respond with the grants. In case of a write request, they write the data from the master to the specified address. For a read request, they send out the requested data on rdata while they set rvalid to 1. This means, each of the ibex cores, the management core, and the UART to mem input can each read from and write to both SRAMs and UART. The inter module is responsible for arbitrating these requests.

The arbiters used in the inter module grant access to the masters in a round-robin fashion. Every one of the three slaves has its own arbiter. A token is granted to the master who gets to access the bus. On reset, master 0 (the ibex core) gets the token first. Then, as long, as the current token holder requests access to the bus, the access is granted. As soon as it does not request the access anymore, the token is passed to the next master who is currently requesting. The tokens circle in the order of the masters, while masters that don't have their request signal high are skipped. If two masters request access to the given slave, the master who is next in order gets granted access. This means that each master needs to make sure it is not blocking any given slave constantly with a high request signal so that the other masters also get a chance to access this slave. In reality, this is not an issue because even if a core is polling a certain SRAM address constantly, it is not requesting the read from SRAM the entire time. The actual request is only active while an assembly load instruction is executed, and since polling a certain address requires a loop in assembly, there are always a few instruction cycles, where the core is busy with the loop logic and not requesting an SRAM read. This time is enough for another master to secure the token. So the polling of an address by a core is not blocking the other masters. Different masters can read from or write to different slaves at the same time, because every slave has its own arbiter that coordinates access to that slave. Different masters cannot read from or write to the same slave at the same time, even if different addresses within that slave are being requested. For reference, the arbiter module is shown in Listing 8 in the appendix.

## 3.4 Masters

The four masters are:

- master 0: core 1, the ibex core
- master 1: core 2, the flexbex core
- master 2: the caravel management core that can interact with the user design through the wishbone bus
- master 3: the UART master module

They will be described further now.

### 3.4.1 The ibex core

Master 0 is the ibex core. Documentation for the ibex core can be found at [17]. It implements the RV32I base integer ISA with the standard extensions M and Zkn, so it implements RV32IMZkn according to the RISC-V naming conventions found in the manual [7]. On reset, its program counter is set to 0x080, which means that it starts executing code in SRAM 1 at address 0x80. Since the SRAM is initially empty, it first needs to be filled with the instructions to be executed by the core. This can be done via UART through the uart_to_mem module or via the management core by modifying its firmware accordingly. As soon as the instructions have been stored in SRAM, the ibex core can be started by asserting its fetch enable signal, which corresponds to the GPIO 5 input. The ibex and flexbex cores can both access any of the slaves through load and store instructions. For example, a store word to address 0x010 writes the value of the register, which is given in this instruction, to address 0x10 in SRAM 1. A load byte from address 0x800 is reading the byte received by the UART peripheral. For further details, see Section 3.5. The eFPGA can be configured from the ibex core. When a certain custom instruction is executed by the latter, the SelfWriteData and SelfWriteStrobe signals for configuring the fabric get set accordingly. This is explained in more detail in Section 3.6.2.

### 3.4.2 The flexbex core

Master 1 is the flexbex core. The eFPGA fabric can only be configured from the ibex core and not the flexbex core. The flexbex core also starts executing instructions at address 0x080, so it starts in SRAM 1 at address 0x80, just like the ibex core. To avoid the two cores always executing the same code, they can be started at a different time by asserting the respective fetch enable, which is the GPIO 7 input for the flexbex core. If both cores should execute separate instructions, the bootcode at address 0x80 needs to be changed after starting the first core to make sure the second core jumps to a different part of the SRAM containing its instructions.

### 3.4.3 The wishbone interface to the Caravel management core

Master 2 is the caravel management core, which is connected to the user project through a wishbone interface. It is not further explained here since it is not used explicitly by the programmer. The mgmt core can be used to load the instructions for the ibex core into SRAM on startup. Its firmware can be written in C. The management core will be further described in Section 3.7.1.

### 3.4.4 The UART to mem interface

Master 3 is a UART module. The uart_to_mem master can request a write to any SRAM or the peripheral slave, which is the UART output. The GPIO 9 input is the rxd input of the uart_to_mem module, through which data can be sent in. The GPIO 11 output is the txd output, through which data is sent out. To send data through the UART to mem module, rxd needs to send the byte WRITE_CMD first, which is 0x41. Of the next byte, the MSbs are 011-, because the first three bits need to match PKT_ADDR[7:5] = 3'b011. The bit after that is a don't care. The LSb nibble determines which slave is requested and the start of the address. The next byte is the end of the address. In total bytes two and three that are sent through UART after 0x41 (the WRITE_CMD) are:
PKT_ADDR[7:5] - SLAVE_SELECT[1:0] ADDR[9:0].
SLAVE_SELECT[1:0] can be 00, 01, or 10. 00 encodes a request to slave 0, which is SRAM 1, 01 is a request to slave 1, SRAM 2, and 10 is a request to the peripheral slave (UART). ADDR[9:0] is the address within the selected slave to which the request is directed. Since the ibex core starts to execute code starting at 128 (0x80), that is the address, the instructions should be written to before starting up the ibex core. Since the core increments the pc by 4 after each instruction (that is not a jump or branch), the instructions should be written into SRAM at addresses 0x80 (128), 0x84 (132), 0x88 (136) and so forth. After the WRITE_CMD and the two byte address, the data to be written can be sent in through UART. That are the next four bytes which then fill up a word in SRAM. To send another word, the process needs to be repeated by sending another WRITE_CMD, then the two bits with the address, and index of the requested slave, and then the four data bytes. An example would be:

```
1) 0x41 = 8'b0100 0001: WRITE_CMD
2) 0x60 = 8'b0110 0000: PKT_ADDR[7:5], -, SLAVE_SELECT[1:0], ADDR[9:8]
3) 0x80 = 8'b1000 0000: ADDR[7:0]
4) 0x00 = 8'b0000 0000: DATA[31:24] (here: instruction 0x00100113: addi x2 x0 1)
5) 0x10 = 8'b0001 0000: DATA[23:16]
6) 0x01 = 8'b0000 0001: DATA[15:8]
7) 0x13 = 8'b0001 0011: DATA[7:0]
8) start again with step 1) for the next word
```

Every byte starts with a UART start bit 0, then the byte is sent LSb first. After the MSb, a stop bit 1 is sent. Then the next byte is sent, preceded by a start bit and followed by a stop bit. This example writes the instruction 0x00100113 (addi x2 x0 1) to address 0x80 in SRAM 1. This can be repeated to load all instructions for the ibex core into SRAM. The baud rate expected by the UART to mem master is 1/8 * the clock frequency.

The RTL code of the UART to mem module uses blocking verilog statements in an always block, which might lead to unexpected behavior of the hardware. There might be a mismatch between the simulation and the synthesized hardware. The hardware might require 9 bits instead of 8 between every start and stop bit. This can be accomodated by simply sending two start bits for every byte that is sent through UART, but might cause unexpected behavior at first, because it does not conform to the usual behavior of UART.

In theory, the UART to mem module can also read from one of the slaves, but because the output enable bits are flipped, all UART outputs are permanently disabled in the ICESOC. This is explained in Section 3.7.2.

## 3.5 Slaves

The three slaves are:

- slave 0: SRAM 1
- slave 1: SRAM 2
- slave 2: peripheral; This is a connection to UART. Through this, the masters can receive data via UART.

They will be described further now.

### 3.5.1 The SRAMs

Both SRAMs have a size of 1KB. A word has 32 bit, so they can each hold 256 words.

The addr output of the ibex core, which determines which address of which slave is requested, is a byte address. Therefore it is incremented by four, every time a new word is accessed. The SRAM is word-addressed through its address input. The byte-addressability is realized through a write mask, the byte enable signal. But the byte-address output of the ibex core is directly passed to the word-address input of the SRAM without converting the byte-address to a word-address and a write mask. This mismatch leads to 3 words being skipped every time the core wants to access the SRAM. This reduces the size of both SRAMs to 1/4th of its actual size because the other 3/4 of the addresses are not reachable by the cores. They have their LSb address bits set to a hard 0, which ensures, the address output and therefore the word address for the SRAM is always a multiple of four. Therefore instead of 256 words, each SRAM can only hold 64 words, which are addressable by the cores. The read-only port of the SRAMs is used for instruction fetching through the inter_read module.

When starting the cores by asserting their fetch enable signal, they both start executing the code at 0x080 in SRAM 1. This needs to be taken into account when placing the instructions in SRAM. So the bootcode needs to be placed starting at address 0x80, which is exactly in the middle of SRAM 1. Except for this, both SRAM are equivalent.

### 3.5.2 The UART peripheral

The UART is memory-mapped to address 0x800, which corresponds to address 0x00 in slave 2, the UART peripheral. A read or write targetet at address 0x804 accesses the UART prescaler, which needs to be set before the UART peripheral can be used.

The prescale value is multiplied by eight and serves as a clock divider. So the bit period of the UART will be $8 * prescale * clk\_period$. The baud rate is the inverse, so $(clk\_frequency)/(8 * prescale)$.

By default the prescale is 0 which leads to a very slow UART since initializing the counter with $(prescale * 8) - 1$ leads to an underflow of 0x7ffff on the 19-bit signal. The bit period will therefore be around $500000 * clk\_per$. So the prescale value needs to be set manually by storing the desired value to address 0x804 before using the UART peripheral.

The input of the UART peripheral, rxd, is the GPIO 8 input, and its output, txd, is the GPIO 10 output. However the UART output is permanently disabled because of the flipped output enables, as explained in Section 3.7.2.

## 3.6 The eFPGA

The eFPGA has three slots, which can each host a custom instruction. Larger custom instructions occupying two ore more slots are also possible.

Fig. 4 shows the eFPGAs connectivity to the cores and to the outside. The Config module on the top left handles the configuration of the eFPGA.

The west ports (W) of the eFPGA are connected to the ibex core, while its east ports (E) are connected to the flexbex core. Each core provides two operands (OPA and OPB), and gets three results back (RES0-2) from the eFPGA. Those are the results of every slot hosting the custom instructions. The operands and results are 32 bits each. In Fig. 4, it can be seen, that all connections between the fabric and the cores have a width of 36 bit. Below the name of the signal, like W_OPA[35:0], an explanation is shown, which bits carry which information. For example the operator, or slot, which is also part of the custom instruction as explained in Section 3.6.1, are the two bits [35:34] of W/E_OPB, while the actual operand B are only the bits [31:0] of W/E_OPB.

The eFPGA fabric can be configured by the ibex core though its SelfWriteData and SelfWriteStrobe signals. In Fig. 4 this can be seen by the signals connecting core 1, the ibex core, to the Config module. The configuration is triggered when the ibex core is executing a custom instruction where the field which specifies the slot is set to 2'b11. Register rs1 then contains a word of the bitstream. When this instruction is executed, the contents of register rs1 are written to SelfWriteData while SelfWriteStrobe is asserted. To configure the entire fabric, the bitstream needs to be loaded word by word into the register specified by rs1. Then the ibex core needs to execute the custom instruction for configuration. This is explained in detail in Section 3.6.2. Alternatively, the configuration could be achieved by sending the bitstream through the UART Rx signal (GPIO 8), which can be seen in Fig. 4 as an input to the Config module on the top left, or through the bitbang interface, also on the top left, by sending the bitstream though s_data (GPIO 4) and providing the corresponding clock through s_clk (GPIO 3).

There is also a 10 bit connection of the eFPGA directly to the GPIOs. On the top right of Fig. 4, the O_top, I_top, and T_top signals can be seen. They connect directly to the GPIO pins 26:17. The ICESOC does not contain a Block RAM, so the FAB2RAM and RAM2FAB interface is unused.

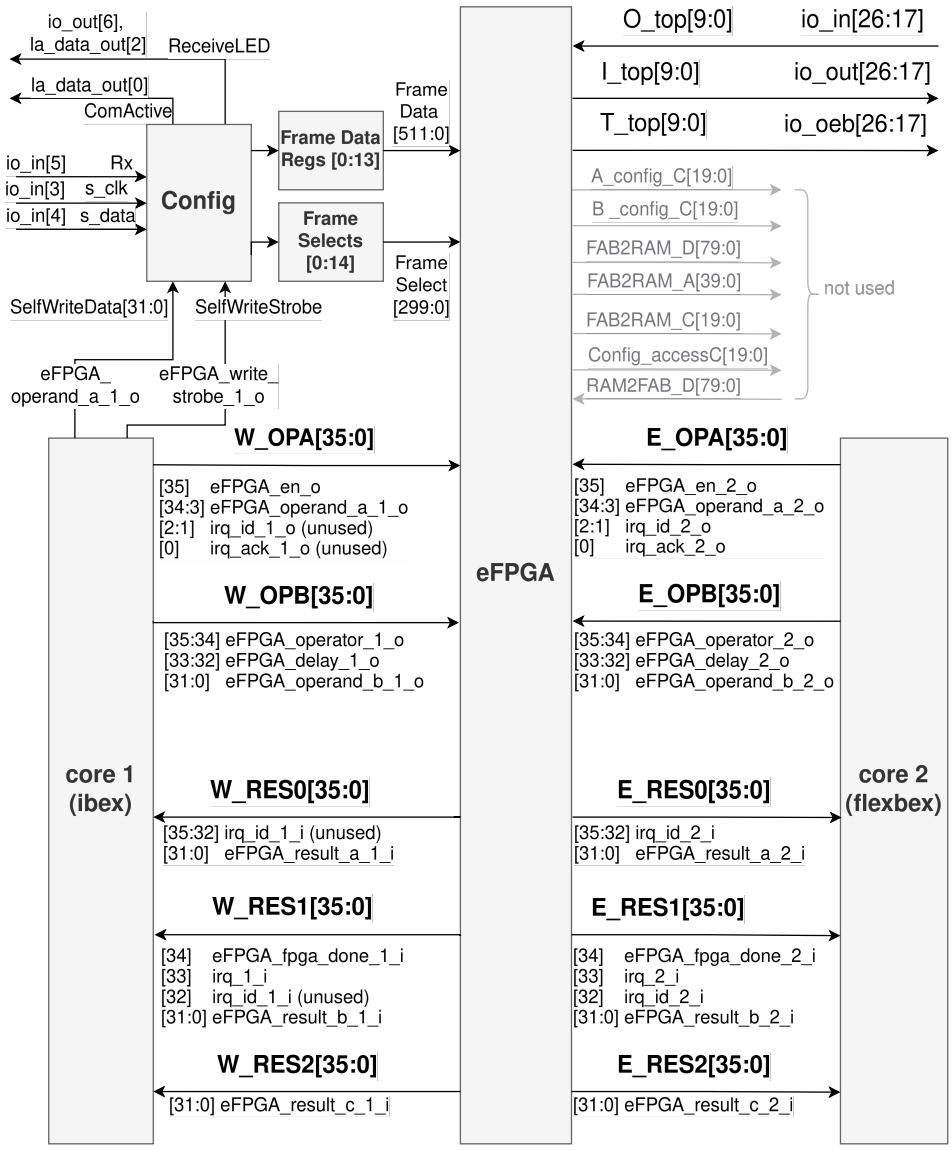


Figure 4: eFPGA connectivity

### 3.6.1 Custom Instruction Format

The instruction that is used to access the custom instructions on the eFPGA uses the RISC-V R-type instruction format with opcode 0x0b. As explained in [1] the syntax of the instruction is

```
eFPGA[result select0-3]d[delay 0-15 cycles] rd, rs1, rs2
```

[result select0-3] is the eFPGA operator, which specifies in which slot of the eFPGA the desired instruction is located. The result of the specified slot is written to the destination register rd of the core. If this field is 2'b11, the fabric is configured with the bitstream word currently in register rs1. This operator field is located in the LSbs of the funct3 field of the R-type instruction, which are bits 12 and 13.
[delay 0-15 cycles] is the delay, which specifies for how many cycles the ibex core should be stalled. The delay field is part of the funct7 field at bits 25-28 of the instruction.
The instruction format is shown in Fig. 5.

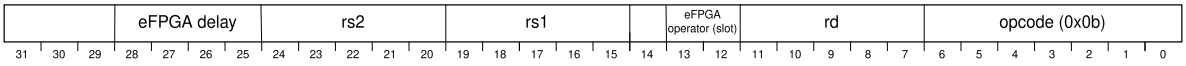


Figure 5: Custom instruction format

Listing 1 shows part of the case statement in the instruction decode module. Only the case which matches the opcode used for the custom instruction, namely 0b is shown. In lines 8 and 9 it can be seen, that the operator are bits 13 and 12 of the instruction, and the delay are bits 28 - 25, as shown in Fig. 5. In lines 5 - 7 the register file read and write enable signals are asserted, just like with the other R-type instructions, like ADD. In line 10, the eFPGA is enabled, and in line 11, the illegal instruction flag is deasserted.

```
1    opcode = instr[6:0];
2    case (opcode)
3        ...
4        7'h0b: begin // opcode for custom instructions
5            rf_ren_a_o = 1'b1;
6            rf_ren_b_o = 1'b1;
7            rf_we = 1'b1;
8            eFPGA_operator_o = instr_rdata_i[13:12]; // slot
9            eFPGA_delay_o = instr_rdata_i[28:25];    // delay
10           eFPGA_int_en = 1'b1;
11           illegal_insn = 1'b0;
12       end
13       ...
14   endcase
```

Listing 1: Part of the case statement in ibex_decoder.v. source: [14] Comments were added.

After the fabric has been configured, the custom instructions on the fabric can be used by issuing a custom instruction which selects which source registers provide the two operands A and B. Those two operands are routed to all three slots. The eFPGA operator field selects which slot's result is written to register rd. Since the execution time of the instructions on the eFPGA can vary, a delay can be specified as part of the instruction. It stalls the ibex core for the specified amount of cycles. This should ensure that the result of the desired slot is ready by the time the next instruction is issued.

### 3.6.2 Configuring the eFPGA

When configuring the eFPGA fabric from the core, a custom instruction with its "eFPGA operator" field set to 2'b11 is used. rs1 is set to the register holding a word of the bitstream. One example for this is the instruction 0x000ff00b. It sets rs1 = t6, the slot to 11 for configuring, and the rs2, rd, and delay fields to 0. When the custom instruction is executed, the contents of register t6 are written to the eFPGA through the SelfWriteData signal while SelfWriteStrobe signals the validity of the data signal with a strobe. Since this only writes a single word of the bitstream from a register to the configuration interface of the eFPGA, the bitstream needs to be loaded into that register sequentially by the core and then sent to the eFPGA by executing the custom instruction every time.

The eFPGA is integrated tightly into the ibex core's pipeline in such a way that it is used as an additional execute block in parallel to the ALU. This results in the execute block of the ibex core containing a module ibex_eFPGA, which handles the interface logic to the eFPGA. A snippet of this can be seen in Listing 2. It shows a finite-state machine (FSM), which controls the write_strobe and endresult signals. In line 7, write strobe is asserted, which corresponds to the SelfWriteStrobe needed

```
1     case (eFPGA_fsm_r)
2         2'd0: begin
3             count <= 0;
4             if (en_i == 1) begin
5                 eFPGA_fsm_r <= 2'd1;
6                 if (operator_i == 2'b11) //if the operator is 2'b11, the fabric will be
                  ↪  configured, so write_strobe is asserted
7                     write_strobe <= 1'b1;
8             end
9         end
10        2'd1: begin
11            count <= count + 1;
12            if (((count == delay_i) & (delay_i != 4'b1111)) | ((delay_i == 4'b1111) &
              ↪  efpga_done_i)) begin //if the delay has passed, or the delay field was set to
              ↪  4'b1111 and the eFPGA done signal is asserted by the fabric, return the
              ↪  result
13                eFPGA_fsm_r <= 2'd2;
14                case (operator_i) //selecting the result of which slot is returned
15                    2'b00: endresult_o <= result_a_i;
16                    2'b01: endresult_o <= result_b_i;
17                    2'b10: endresult_o <= result_c_i;
18                    2'b11: begin //configuring
19                        endresult_o <= result_a_i;
20                        write_strobe <= 1'b0;
21                    end
22                    default: endresult_o <= result_a_i;
23                endcase
24            end
25        end
26        2'd2: eFPGA_fsm_r <= 2'd0;
27        default:
28            ;
29    endcase
```

Listing 2: The FSM in ibex_eFPGA.v. source: [14]. Comments were added.

to configure the fabric. This is triggered by an operator, which has the value 2'b11, as explained above. Other than controlling the SelfWriteStrobe signal, the FSM also controls which of the three results from the slots will be selected. The FSM keeps incrementing a counter until as many clock cycles have passed as specified in the delay field of the instruction. If the delay field specifies a delay of 4'b1111, the FSM waits until the fabric returns a done signal. When the delay has passed or the fabric has signaled that it is done, one of the three results is selected depending on the slot/operator field of the instruction, as seen in Fig. 5. When configuring with the slot field having the value 2'b11, write strobe is deasserted as soon as the specified delay has passed, as seen in line 20 in Listing 2.

## 3.7 Caravel

The Caravel harness contains the user project, or mega project (mprj) area, which contains the ICESOC explained above. The user project interface to the Caravel harness consists of a wishbone interface, logic analyzer signals, and 38 GPIO ports.

### 3.7.1 Caravel management core

The Caravel harness contains a management RISC-V core, which is a VexRiscv core. Its firmware is located in the file wb_test_icesoc.c and can be modified as desired. The mgmt core is responsible for configuring the GPIOs by selecting which GPIO mode should be used. For every GPIO pin, a register reg_mprj_io_"pin_nr" contains the pad configuration. Those registers can each be set to a GPIO mode. Some of the GPIO modes that can be selected are:

```
GPIO_MODE_MGMT_STD_OUTPUT,
GPIO_MODE_MGMT_STD_INPUT_PULLDOWN,
GPIO_MODE_USER_STD_BIDIRECTIONAL,
GPIO_MODE_USER_STD_OUTPUT.
```

Selecting a pin as mgmt output disconnects the user output and routes the corresponding GPIO output of the mgmt core through to the pads. To enable inputs and outputs to the user project, the GPIO_MODE_USER_STD_BIDIRECTIONAL needs to be selected. This is done for example for the pins 17 - 26, because those are the I_top, T_top and O_top signals, that are connected to the eFPGA. The T_top signal determines if a given pin is an input or output. It can change dynamically from the user project. The other pins are fixed to either user input or output, or management input or output.

The mgmt core can read from and write to the SRAMs in the user project by using the wishbone bus. This is done by reading from or writing to the reg_mprj_slave register, which points to address 0x30000000, where the user project is located. To access the SRAMs, pointer are defined as follows:

```
volatile uint32_t *sram1 = (uint32_t *)&reg_mprj_slave;
volatile uint32_t *sram2 = (uint32_t *)&reg_mprj_slave + 0x100;
```

This means, SRAM 1 is located starting at address 0x30000000, while SRAM 2 is at address 0x30000000 + 0x100 * sizeof(uint32_t), so at address 0x30000400. Similarly, the UART peripheral would be mapped to 0x30000800. In the firmware, a write to SRAM 1 with "sram1[0] = value" corresponds to a write to the first accessible word in SRAM 1.

The mgmt core can communicate with the user project through a wishbone interface.

### 3.7.2 The GPIOs

The user project has 38 GPIOs, which are described in Table 1. They are bidirectional, so every pin can act as an input or as an output depending on how it is configured. With the oeb signal, which is the low active output enable signal, the user project can decide if a certain pin should be an output (oeb=0) or an input (oeb=1). Since it is fixed to 1 for the Txd UART signals, they are permanently disabled and no UART output is possible out of the user project. This cannot be fixed by the management core, since it can overwrite the output enable signal, but that only enables the output of the management core and overwrites the output of the user project. Additionally, the management core can disconnect the user project GPIO signals and instead connect its own GPIO inputs or outputs to the pads. This can be configured in the firmware for the mgmt core by setting the reg_mprj_io for every pin to its desired mode, as described in Section 3.7.1.

| | **io_in** | **io_out** | **io_oeb** |
|---|---|---|---|
| 0 | clk | - | 0 |
| 1 | clk_select[1] | - | 0 |
| 2 | clk_select[0] | - | 0 |
| 3 | s_clk & la_select | - | 0 |
| 4 | s_data & (if la_select == 0) debug_req_1 | - | 0 |
| 5 | Rx & (if la_select == 0) fetch_enable_1 | - | 0 |
| 6 | (if la_select == 0) debug_req_2 | ReceiveLED | 0 |
| 7 | (if la_select == 0) fetch_enable_2 | - | 0 |
| 8 | rxd_uart | - | 0 |
| 9 | rxd_uart_to_mem | - | 0 |
| 10 | - | txd_uart | 1 |
| 11 | - | txd_uart_to_mem | 1 |
| 12 | - | error_uart_to_mem | 1 |
| 13 | - | - | - |
| 14 | - | - | - |
| 15 | - | - | - |
| 16 | - | eFPGA_write_strobe_2_o | - |
| 17 - 26 | O_top | I_top | T_top |
| 27 - 37 | - | - | - |

Table 1: The user project io pins.

# 4 Firmware for SRAM paging

## 4.1 Motivation

The ICESOC has two 1KB SRAMs in its user project area, which can therefore store $1KB/4 = 256$ words each. The Ibex core assumes the SRAM to be byte-addressable with its address signal. But the SRAM actually is word-addressed by its address input and has a separate byte-enable signal to select the bytes within a word. Because of this mismatch, only every fourth word is addressable by the cores. So the SRAM effectively shrinks to $256/4 = 64$ words per SRAM. Both SRAMs combined can therefore store 128 words. When dividing this into stack, maybe a heap and an instruction memory, it becomes clear that programs, which fit into the instruction memory and therefore consist of less than 128 instructions would be very limited. In addition, if the eFPGA fabric should be reconfigured from the core, the bitstream needs to be passed through the SRAM to the core. The full bitstream contains 4506 words, which obviously do not fit into the SRAM.

For those reasons, a paging mechanism is implemented here, which loads the bitstream and the instructions piece by piece into the SRAM and processes them accordingly. Both SRAMs will be split into physical pages, whose number and size can be changed as desired. For controlling those parameters as well as selecting the program that will be executed by the ibex core and the bitstream to configure the fabric, a header file is used. When modifying the program, one needs to make sure to match the parameters specified in that header file. At least two pages should be used. Otherwise the possible latency hiding of paging, meaning that the next page can be loaded before the current one is finished, cannot be taken advantage of.

Here, the RISC-V assembly code for the ibex core was written by hand to make sure it can be split up into pages with exactly the desired size.

## 4.2 Communication and synchronization between the cores

If multiple cores work together, there needs to be some form of communication and synchronization between them. Every core runs a process, and it needs to be made sure, those processes don't interfere with each other in an unwanted way. There are a few commonly used mechanisms which will shortly be introduced here.

A critical region is a part of a program, which accesses shared memory [18]. The access to this critical region needs to be controlled, so no two processes enter it at the same time, because they could perform colliding writes to the shared memory. To ensure this, mutexes or semaphores could be used.

A mutex is a shared variable, which can be either in state 0 or in state 1, which correspond to unlocked or locked [18]. When a process wants to access a critical region or shared resource, it must obtain the mutex. The mutex can only be obtained if it is unlocked (in state 0). Then the process obtaining it sets it to 1 and can access the critical region now. When it is done it sets it to 0 again, so another process can obtain it. This ensures, the shared memory is only modified when no other process is currently accessing that part of the memory, therefore ensuring **mut**ual **ex**clusion of the two processes.

A semaphore works in a similar way, but can allow multiple processes to access the resource it is controlling by allowing positive values other than 1, which allows decrementing it more than once by multiple processes.

Communication between processes can be done through message passing. A mailbox is essentially just a buffer for those messages [18]. This could be a part of the shared memory, where the messages are written until that buffer is full. When the mailbox is full, the producing process needs to wait until

it can write a new message. When it is empty, the consumer needs to wait for a message to appear until it can read it. Two semaphores can be used to coordinate access to the mailbox. One of them coordinates how many messages can be written to the mailbox at any given time, the other one signals how many can be read.

The ICESOC consists of multiple cores, the ibex and flexbex cores inside the user project, as well as the management core in the Caravel harness. In the following, only the management and ibex cores are considered, since the flexbex core will be deactivated for now. But the management core and ibex core are both active at the same time and therefore need to be synchronized.

The management core should provide the program for the ibex core and the bistream for configuring the fabric by writing it to the user project SRAM. So the management core provides data for the ibex core. The ibex core just reads this and does not need to provide any data. So the management core is the producer, and the ibex core the consumer of the data. Since the SRAM's size is limited, not all data can be written at once, but in pages that are written sequentially.

In this scenario, synchronization is needed to prevent the producer or consumer from being too fast and either overwriting data pages that have not been consumed yet or trying to consume pages that have not been produced yet.

Since the hardware is fixed, no additional components, like a hardware FIFO can be added. Therefore the desired mechanism needs to be implemented in the firmware of the two cores. Since both cores can access the SRAM in the user project, they can communicate through this shared memory.

The task here is to coordinate two cores, one of which only produces, and one of which consumes the data pages, in a system with shared memory.

This synchronization can be achieved, by introducing a request and a ready signal to establish a simple handshake between the cores. The ibex core requests the next data page, as soon as it has processed a page that can be overwritten. The management core needs to wait for a request, then provides the requested data page, and signals that the data is ready. The ibex core only consumes pages which are available as signaled by the ready signal. Those request and ready signals are implemented as shared variables. They are located at a certain address in SRAM. Each variable is only written by one core, while the other is just reading it. Each shared variable is a counter which holds the index of the page which is requested or ready. Counting up the index avoids timing problems, because the variable is not just toggling a value, which would be sensitive to timing issues. One drawback of using this counter is, that it limits the maximum possible number of pages to the largest counter value representable by the 32-bit word in which the counter is stored. The biggest possible counter value is $2^{32}$, which is around 4290 million. Since the bitstream for the entire fabric contains 4506 words, this limit will not be reached for the bitstream paging. For the instructions it is assumed that $2^{32}$ instruction pages will be enough. But this limit needs to be kept in mind for larger applications.

The shared variables are always only written by one core and read by the other. The actual data pages are written only by the management core and read by the ibex core. So there can be no write conflicts of the two cores trying to write to the same memory address. Therefore no mutex or semaphore is necessary to coordinate access to a certain memory address directly. But the underlying data structure is essentially a mailbox. The messages written to the mailbox are the instruction or bitstream pages, which are written to specific areas in SRAM in a FIFO fashion. The shared variables coordinate access to the FIFO, and function almost like semaphores. The request variable indicates the maximum page index that can be written before the FIFO is full. The ready variable indicates the maximum page index that can be read before the FIFO is empty. If the shared variables were not

continually incremented, but had a maximum value matching the number of physical pages, they would work like a conventional semaphore. Since the pages are loaded sequentially and deterministically, this simple solution works.

In this scenario, only two cores are active and both respect the memory layout. It is not secure, each of the cores could write to the variable it is only supposed to read. Also the flexbex core could be enabled as a third core. It would have to be coordinated with the other two active cores.

This implementation is only needed for the simple case of configuring the eFPGA fabric with a bitstream passed from the mgmt core to the ibex core and then providing the instructions for the ibex core. It fulfills this one purpose, which should be sufficient.

With shared memory, cache consistency is another important aspect, which does not apply here, since both cores used do not have a data cache.

### 4.3 Implementation details

The implementation of the SRAM paging mechanism consists of the firmware for the management core in wb_test_icesoc.c, and the program for the ibex core in program.s.

The communication between the mgmt core and the ibex core works over shared variables in SRAM. Those variables are located at the beginning of SRAM 1, as can be seen in Fig. 6. For bitstream and instruction paging, there are two variables each. The request variable is written by the ibex core. It is a counter, which holds the index of the page that is requested. The management core only reads this value and provides the requested pages. The ready variable is written by the management core. It is also a counter, that signals the index of the last page, which has fully been written and is therefore available to the ibex core. The ibex core only reads this value to check if a new page is available or not.

The parameters for the size and number of physical instruction and bitstream pages are located in the header file instr_bitstr_sizes.h. The physical number of pages refers to the maximum number of pages of a certain size, that the section in SRAM, predestined for the paging, can hold at the same time. The size is the number of instructions per page. In this implementation, the entire SRAM 2 is used for instruction paging, so its 64 words can be split up into pages as desired. As stated above, at least two pages should be used. So one possibility for the instruction page parameters is using a size of 32 instructions per page, and two physical instruction pages. Another option would be to use four physical pages of 16 instructions each. So when choosing those parameters, they need to fulfill $instr\_page\_size * nr\_instr\_pages = 64$.
For the bitstream paging, separate parameters are used, since the bistream is paged in the second half of SRAM 1. So in total, only 32 words are available for the bistream pages. So the biggest sensible size for a bitstream page would be 16 words per page. From this follows, that the bitstream page parameters need to fulfill the constraint $bitstr\_page\_size * nr\_bitstr\_pages = 32$.

As explained above, the designated space for bitstream or instruction paging is split up into as many pages of a certain size as specified with the parameters. Each page has a logical page index, which is the page_counter that is stored in register s3 for the instruction paging as shown in Table 2. For the bitstream paging, it is stored in a temporary register. This is a running index over all pages. Each page also has a physical page index which corresponds to the index of the physical page the logical page will occupy. The physical index is in the range between 0 and nr_physical_pages - 1. It is also called the page_counter_modulo, since it is just the page_counter modulo the number of physical pages. For the instruction paging, register s4 is used for this.

The management core and the ibex core each independently keep track of the start address of the

current and next pages, which depend on the physical page index. To get the next page start address from the current start address, it has to be incremented by the size of a page. If the last physical page has been reached (physical index = nr physical pages - 1), the start address is reset to the start address of the memory area reserved for the paging, which is the first physical page, and the physical index is reset to zero.

The program for the ibex core is split up into pages, which are loaded into SRAM sequentially. The first three pages are called page A, B, and C and are needed to initialize the paging mechanism and to configure the eFPGA with the bitstream. They have a fixed size and should not be modified. After page A - C, the pages have a numeric index starting at 1. Page 1 is the first customizable page, which can execute any user program. The numeric pages can be modified to execute any program as needed. Certain aspects have to be kept in mind when modifying those pages, which will be explained in Section 4.3.4.

This implementation is kept relatively minimal and not a lot of checks are done in the code. Since extra checks would require more instructions and therefore more space in SRAM, which is scarce, certain assumptions are made which are not checked in the firmware. So if for example the page number and size don't match, this will result in unexpected behavior and correct usage needs to be ensured manually.

As a short overview, the SRAM paging works as follows. First, the mgmt core writes the bootcode for the ibex core into SRAM 1, to address 0x80, at which the ibex core starts looking for its instructions. When started, the ibex core executes those instructions, which includes requesting new instruction pages as soon as it has finished executing the current ones. When a new page is requested, the mgmt core loads it into SRAM. Loading the bitstream into SRAM and reading it with the ibex core works the same way. The ibex core requests the bitstream pages, and the management core then writes those bitstream pages into SRAM. The ibex core reads them and executes a custom instruction, which configures the eFPGA with those bitstream words. The ibex core requests more bitstream pages until the entire bitstream has been sent to the eFPGA and the latter has been fully configured. Then the normal instruction paging is started, where the ibex core executes its instructions page by page. After a page has been executed, the ibex core requests the next page, which the management core will write to SRAM.

This can be split into three phases. Pase 1 is the initialization phase. Phase 2 is the bistream paging, and phase 3 is the instruction paging. They are executed one after another and explained in more detail in the following sections.

### 4.3.1 Phase 1: Initialization

In phase 1, the management core initializes the SRAM with the data needed by the ibex core. The memory layout of the this phase is shown in Fig. 6.

The mgmt core writes the parameters from the header file, like the instruction and bitstream page number, - size, and number of words in the bitstream, to SRAM 1. It initializes the instruction and bitstream page ready and request counters in SRAM 1. Then it writes instruction page A to SRAM 1 at address 0x80, and page B to SRAM 2 at address 0x0. Next it signals to the testbench that the ibex core should be started by setting reg_mprj_data1 to 0x30000000. The testbench sets fetch_enable of core 1 to 1 and thus starts the ibex core.

When the ibex core is enabled, it starts executing the code at address 0x80, which contains page A, the blue area in Fig. 6. It contains code for initializing the stack pointer to 0xac, and loading the parameters that the management core wrote to SRAM before into the ibex core's registers s0, s1, a0,

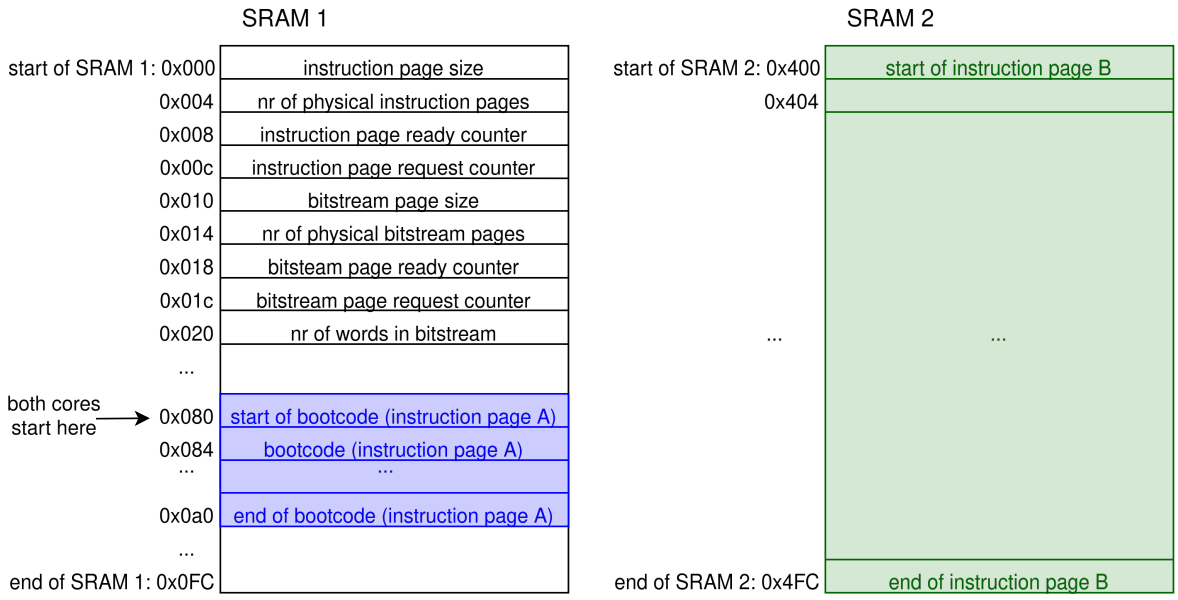


Figure 6: The memory layout of phase 1, the initialization.

a1, and a2. Then the ibex core will jump to page B, which handles the bitstream paging, and page A can be overwritten from now on.

### 4.3.2 Phase 2: Bitstream paging

In phase 2, the bistream is written into a mailbox in SRAM 1 page-wise by the mgmt core. The ibex core consumes those pages and configures the eFPGA with the bistream. The memory layout of this phase can be seen in Fig. 7.

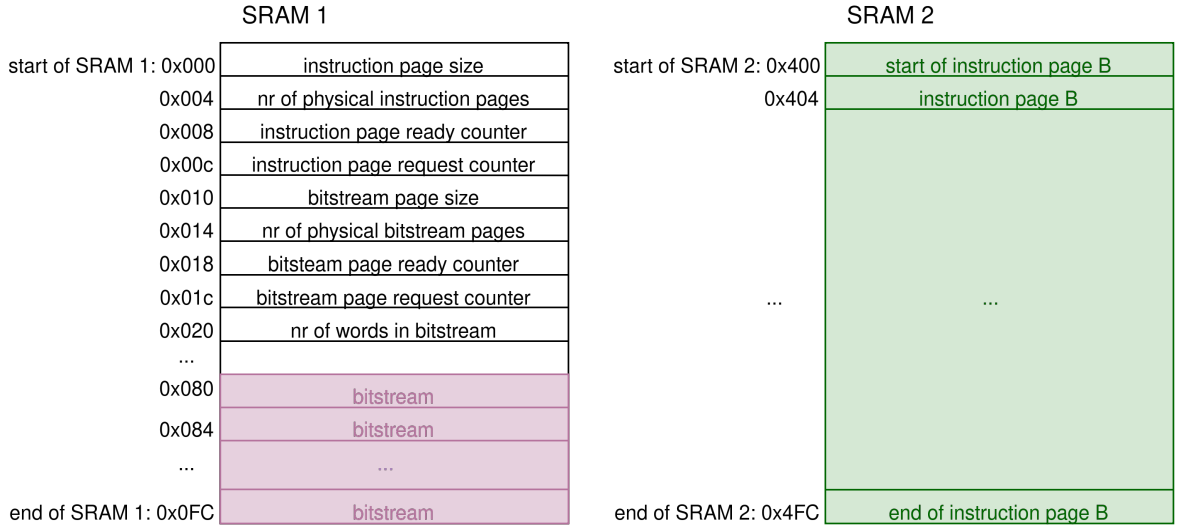


Figure 7: The memory layout of phase 2, the bistream paging.

Here, the cores are synchronized with the bitstream_page_request_counter and bitstream_page_ready_counter shared variables at address 0x1c and 0x18 in SRAM 1, as seen in Fig. 7. This was already described in Section 4.2. The ibex core starts executing instruction page B, which is located at address 0x400,

the start of SRAM 2. Page B is 64 words large and fills the entire SRAM 2. It is represented as the green area in Fig. 6. Page B initially requests as many bitstream pages as there are physical bitstream pages, so as many pages as possible are loaded in advance. The bitstream is loaded into the second half of SRAM 1, which is the pink area in Fig. 7. It will overwrite page A, which was located in this area before but is not needed anymore now. The requesting of the bitstream pages is done by writing the value bitstr_nr_pages-1 to mem[0x1c], which is the bitstream page request counter. The written value is the index of the last page which is requested, so the management core can write all pages with index lower or equal to the requested index to SRAM. Next, the ibex core initializes the variables which keep track of the address to which the next page should be loaded. Then the shared variable bitstream_page_ready_counter is polled until a bitstream page is available. As soon as a page is ready, the bistream is read word by word. For every word, a custom instruction is executed that configures the fabric with that bitstream word by setting the SelfWriteData and SelfWriteStrobe signals accordingly as explained in Section 3.6.1. Once the entire page has been read, the next bitstream page is requested. The start address of the next page is calculated, and the ready counter is polled until that page is available.

```
1     while (word_ctr < BITSTREAM_WORDS) {
2         page_ctr = word_ctr / (PAGE_SIZE_BITSTR/4);
3         page_idx = page_ctr % BITSTR_NR_PHYS_PAGES;
4         sram1[32 + (page_idx * (PAGE_SIZE_BITSTR/4)) + (word_ctr % (PAGE_SIZE_BITSTR/4))] =
          ↪  bitstream[word_ctr];
5         __asm__ volatile("" ::: "memory");
6         if ((word_ctr + 1) % (PAGE_SIZE_BITSTR/4) == 0) { // a full page has been written
7             __asm__ volatile("" ::: "memory");            // to make sure all writes to sram
              ↪  above are done before continuing, otherwise instructions might be reordered
8                 sram1[6] = page_ctr;                      // page ready counter
9             __asm__ volatile("" ::: "memory");
10            page_request_idx = sram1[7];
11            while (page_request_idx < (page_ctr + 1)) {
12                page_request_idx = sram1[7];              // poll page request until next page
                  ↪  is requested
13            }
14        }
15        word_ctr++;
16    }
17    __asm__ volatile("" ::: "memory");
18    sram1[6] = page_ctr;
```

Listing 3: The part of the management core's firmware that handles the bistream paging.

Meanwhile the mgmt core polls the bitstream_page_request shared variable. When the ibex core requests a bitstream page, the mgmt core writes it to the second half of SRAM 1. This can be seen in line 4 in Listing 3, which shows a snippet of the mgmt core's firmware. The address to which the word should be written depends on the current page index and the word index. After a page has been written completely, the index of the completed page is written to the bitstream_page_ready_counter variable in SRAM in line 8. The last page could also be a partial page, the ready counter would be incremented to its index anyways in line 18. Before and after every write to SRAM where the order needs to be preserved, a memory barrier

```
__asm__ volatile("" ::: "memory");
```

is used, to keep the order intact. This is needed to ensure the page ready counter is only incremented after the page has been written completely.

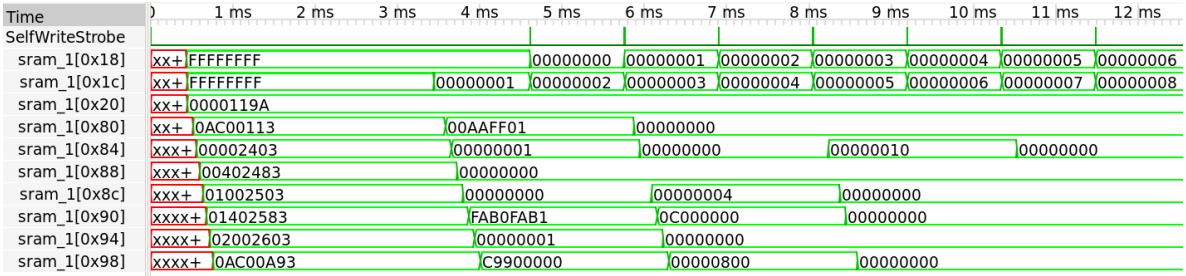


Figure 8: Simulation waveforms of the bistream paging. SelfWriteStrobe signals when a new page is read from SRAM and configured to the eFPGA. Every strobe seen here are actually 16 strobes when zoomed in, because there is one strobe for every bitstream word. The memory layout is shown in Fig. 7. sram_1[0x18]: bitstr. page ready ctr, sram_1[0x1c]: bitstr. page request ctr, sram_1[0x20]: nr words in bitstr., starting at sram_1[0x80]: bitstr. page.

Fig. 8 shows the bitstream paging in the waveforms of a simulation. The SelfWriteStrobe signal is asserted every time a bitstream word is configured to the eFPGA. Every visible strobe here is an entire page that is written, when zooming in, those are actually 16 distinct strobes. Sram_1[0x18] is the bitstream_page_ready_counter, which is incremented by the mgmt core every time a new page has been written. Its initial value is 0xffffffff. Sram_1[0x1c] is the bitstream_page_request_counter, which is incremented by the ibex core when a new page is requested. It had also been initialized to 0xffffffff by the mgmt core. Sram_1[0x20] is the nr_bitstream_words parameter as shown in Fig. 7. The first physical bistream page starts sram_1[0x80]. Only a part of the page is shown as an example. The periodic overwrites of the physical page with the next logical page can be seen here. Once a page has been written fully, the ready variable is incremented by the mgmt core. In response, the ibex core triggers configuration of the eFPGA by asserting SelfWriteStrobe and setting SelfWriteData to the next bitstream word. SelfWriteData is not shown here since it toggles too much to see any relevant value on this scale.

### 4.3.3 Phase 3: Instruction paging

In phase 3, instruction page C, which will permanently be kept in SRAM 1 is written there by the mgmt core. Then the instruction paging is started. The memory layout of the this phase is shown in Fig. 9.

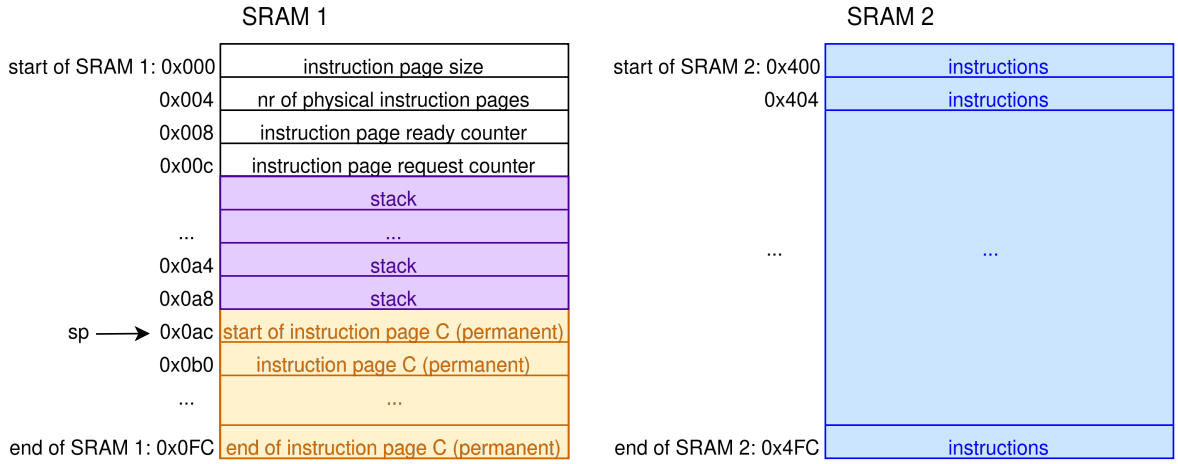


Figure 9: The memory layout of phase 3, the instruction paging.

After the bistream has been read completely by the ibex core and the eFPGA configured with it, instruction page C is loaded into the bottom of SRAM 1, starting at address 0xac. Page C is the yellow area in Fig. 9. Listing 4 shows its code. That page will permanently be kept in SRAM 1. It contains two functions that are needed in every other instruction page, so they will all call those functions in SRAM 1 to avoid the redundancy of every page implementing this functionality itself. The first function req_next starts in line 4 in Listing 4. It requests the next instruction page by incrementing the instruction_page_request_counter. The second function paging starts in line 12. It calculates the address of the next page, polls the instruction_page_ready_counter to check if it is already available in SRAM, then jumps to it. Both of those functions store the values of the register they use on the stack. This way the instruction pages calling them do not need to handle this. An exception are the registers s0 - s4. They store values relevant for the paging functionality and can never be overwritten without breaking the paging functionality. The register usage is shown in Table 2.

```
1  #### page C ########################################################  @0x0ac in mem
2  # this page is written to mem[0xac]=sram1[0xac] and is kept there permanently because every
   ↪  page needs to call the functions req_next and paging
3  # the values of s0, s1, s2, s3, s4, s5 are preserved between pages (don't use them except for
   ↪  the paging)
4  req_next:       addi sp, sp, -4
5                  sw t0, 0(sp)            # store t0 on the stack
6                  lw t0, 0xc(zero)        # read page request counter
7                  addi t0, t0, 1          # increment page request counter
8                  sw t0, 0xc(zero)        # store new page request counter
9                  lw t0, 0(sp)            # restore t0 from the stack
10                 addi sp, sp, 4
11                 jalr zero, 0(ra)
12 paging:         addi sp, sp, -4
13                 sw t0, 0(sp)
14                 addi s4, s4, 1          # s4: page counter modulo++
15                 addi s3, s3, 1          # s3: page counter++
16                 add s2, s2, s0          # s2: start address of next page; start_page +=
                   ↪  page_size
17                 bne s4, s1, poll_instr  # if modulo counter has reached last page, start at
                   ↪  first page addr again and reset modulo counter
18                 addi s4, zero, 0
19                 addi s2, zero, 0x400
20 poll_instr:     lw t0, 0x8(zero)        # instr_page_ready=mem[0x8]
21                 blt t0, s3, poll_instr
22 next_page:      lw t0, 0(sp)
23                 addi sp, sp, 4
24                 jalr zero, s2, 0        # move PC to start address of next page
```

Listing 4: Instruction page C. It consists of the two functions "req_next" for requesting the next instruction page, and "paging" for calculating the start address of the next page, checking if it is available and then jumping to it. This page C will be kept in SRAM 1 permanently starting at address 0xac.

The stack pointer (sp) starts at address 0xac=172, which is the first address of the permanent instruction page C, but it will be decremented before writing a value so the first value on stack is stored to address 0xa8. If it gets large enough, it can overwite the bistream paging parameters, because by the time the stack is used, the bistream paging is finished. The stack is the purple area in Fig. 9 and can be used by the program implemented in the customizable instruction pages. Those instruction pages are located in SRAM 2, as shown with the blue part in Fig. 9. The first page will overwrite instruction page B due to a lack of space, but this is no problem, because only a part that

has already been executed is overwritten.

After page C has been written to memory, the instruction paging is started. This is the mode in which the actual program in the customizable instruction pages will be executed. Initially, the ibex core requests as many instruction pages as there are physical pages available. Next it polls the instruction_page_ready_counter until it is at least 1, then jumps to next instruction page 1, which is the first one that can execute custom code. An example of a customizable instruction page is shown in Listing 5. When calling a function, the register values are not saved on the stack because of the limited SRAM space for instructions. Instead it is made sure manually that no register is overwritten while it is still in use.

```
1  #### page 1 ########################################################  @0x400 in mem
2  # have to pad pages with nops to have a length of exactly page_size
3  main1:          jalr ra, s5, 0x0    # function call of req_next in page C at addr s5 = 0xac
4                  jal ra, cis1        # do stuff here (for example call the function cis1
                   ↪  defined below)
5                  nop                 # do stuff here
6                  nop                 # do stuff here
7                  jalr zero, s5, 32   # function call of paging at addr s5 + 32 (8 instructions
                   ↪  in req_next * 4)
8  cis1:           addi t0, zero, 0xaa
9                  addi t1, zero, 0xf0
10                 nop                 # 0x0053038b  (eFPGA0d0 t2, t1, t0)
11                 nop                 # 0x00531e0b  (eFPGA1d0 t3, t1, t0)
12                 nop                 # 0x00532e8b  (eFPGA2d0 t4, t1, t0)
13                 sw t2, 0x3c(zero)   # store W_RES0 in memory at 0x3c
14                 sw t3, 0x40(zero)   # store W_RES1 in memory at 0x40
15                 sw t4, 0x44(zero)   # store W_RES2 in memory at 0x44
16                 sw t0, 0x48(zero)   # store W_OPB in memory at 0x48
17                 sw t1, 0x4c(zero)   # store W_OPA in memory at 0x4c
18                 jalr zero, 0(ra)
```

Listing 5: Example for a custom instruction page with page size 16. Line 3 always needs to be executed at the beginning of every page. It calls the function in page C, which requests the next page. Line 7 calls the function in page C, that checks if the next page is available yet and jumps to it. Between those two instructions, anything can be done. Here a function cis1 is called, which loads the operands 0xaa, and 0xf0 into two registers and then executes three custom instructions. They are represented as nops and have to be substituted with the values commented out manually if the GCC has not been modified to recognize and compile the custom instructions.

The first instruction in each page should be a call to the request function in page C, as seen in line 3. This increments the page request counter, to request a new page, since any previously finished page can be overwritten now. That way the management core can start writing the next page, while the ibex core is still executing the current one. Then the "useful" code in the page is executed. The last instruction in each page should be a call to the the paging function in page C, like in line 7, which calculates the address of the new page, and polls the page ready counter to see if the next page is available. As soon as the next instruction page is ready, the ibex core jumps there. The last instruction page could call the paging function, like all other pages. Then the next page will be requested. Since the mangement core is not providing new instruction pages, the ibex core will just infinitely poll the ready counter to check if the next page is available. The core could be disabled again by deasserting its fetch_enable input.

Meanwhile the mgmt core starts by checking the page request counter. Every time a new page is requested by the ibex core, the mgmt core provides it by writing it to memory and incrementing

the page ready counter. This is repeated until the entire program from program.s has been loaded into SRAM. When all instruction pages have been provided, the mgmt core sets set reg_mprj_datal to 0x40000000, which the testbench will notice and end the simulation. Before this, a check can be done to return a success or fail message. This can read any value from SRAM and compare it to the expected value if the program was executed correctly.

Fig. 10 shows the instruction paging in the waveforms of a simulation. Here, the instruction pages 1 and 2 contain the CRC application, which will be introduced in Section 5. Page 1 contains the software-only version and stores its result in sram_1[0x10], and the number of clock cycles it took in sram_2[0x14]. Page 3 contains the accelerated version using the custom instruction on the eFPGA. Its result is written to sram_1[0x18], and the number of clock cycles needed is written to sram_1[0x1c]. The input data was the 16 byte string "Hello world!! :D". The resulting CRC checksum is correctly calculated as 0x006726dc for both implementations. As intended, the accelerated implementation achieves a speedup, which will be further discussed in Section 5. The instruction page ready counter is located at sram_1[0x8] and is incremented every time the mgmt core has finished writing an instruction page to SRAM. The instruction page request counter is at sram_1[0xc]. It is incremented by the ibex core every time that a new instruction page is requested. Both variables had been initialized to 0x00000000 by the mgmt core.

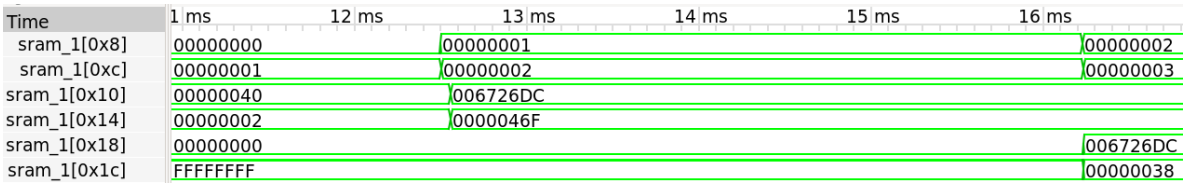


Figure 10: Simulation waveforms of the instruction paging with the result of the CRC application. The memory layout is shown in Fig. 9. sram_1[0x8]: instr. page ready ctr, sram_1[0xc]: instr. page request ctr, sram_1[0x10]: CRC res software-only, sram_1[0x14]: nr clk cycles software-only, sram_1[0x18]: CRC res accelerated, sram_1[0x1c]: nr clk cycles accelerated.

### 4.3.4 User guide

When writing a program and designing a custom instruction for the ICESOC, the SRAM paging implementation can be used. It handles the configuration of the eFPGA and provides the instructions for the ibex core. For this, the new bitstream and the nex program for the ibex core need to be provided. A few things need to be kept in mind for this.

The configuration of the eFPGA can either be simulated completely using the bitstream paging, which takes a long time since the bistream pages have to be written to memory sequentially, or it can be emulated by setting the eFPGAs configuration memory to the desired values right at the start of simulation. This is a lot faster, since the bitstream paging is completely skipped. For testing the actual chip, the bistream paging would still be needed.

For the emulation approach, the number of words in the bitstream has to be set to zero in the instr_bitstr_sizes header, as mentioned before. The bitstream has to be provided as a .vh file, which is generated by FABulous when generating the bitstream. Then the -D EMULATION flag has to be provided to iverilog together with the .vh file.

To simulate the bitstream paging, the number of words in the bitstream would be set to 4506, which is the total number of words in the bitstream. The bitstream is included in the instr_bitstr_sizes.h as a header file, which is generated from the hex file using a python script.

When adding a user program to the numeric pages, certain rules have to be followed. Firstly, the instruction page size specified in instr_bitstr_sizes.h needs to match the number of instructions per page in the ibex program. Also the total number of instructions has to be adapted to match the number of instructions in the numeric pages. So if two numeric pages are used, the total number of instructions should be set to 2 * page size. When changing the program, the first three pages A, B, and C which are marked as such with comments should not be modified. They are responsible for configuring the eFPGA with the bistream and initializing the instruction paging mechanism. After page C, pages 1, 2 etc. can be filled with a custom program. However, each page needs to start with the instruction

```
jalr ra, s5, 0x0,
```

which calls the req_next function in instruction page C, as explained in Section 4.3.3. Also, every page needs to end with the instruction

```
jalr zero, s5, 32,
```

which calls the paging function in page C.

Every label used in program.s needs to be unique for correct compilation. For now, every label has the index of its page appended to it. As long as labels don't repeat, this can theoretically be handled otherwise.

The code in program.s needs to be split into pages, which are separated by comments for clarity. The comments are ignored when compiling the code, so the programmer manually needs to make sure that the pages all have the correct size. The pages have no knowledge of each other, so they all call the next one but never go back or dynamically call another page. The code is essentially executed sequentially page by page. Every page needs to be standalone. In theory, a page can pass values to the next one through registers, or on the stack. That this works as expected needs to be ensured by the programmer.

Every page except for the first three pages A, B, and C, has to contain exactly the number of instructions that match the page size specified in the header, since the instructions are counted and loaded into SRAM page-wise this way. So if less instructions are needed than that page size, the page needs to be padded with nops.

The paging mechanism only works, because the variables needed, like the address where the next page will be located, are stored in registers s0-s5, which keep their value between the pages. This means that these six registers can never be used for anything except the paging mechanism when modifying program.s. Or if they would be used, their state would need to be saved to the stack before modifying it, and restored before calling the paging function at the end of every page. This can be seen in Table 2, which shows the register mapping for the program executed by the ibex core. Registers a0-a2 are used for the bistream paging, but they are not needed anymore by the time the user program starts to execute, because the bistream paging is finished at that point.

When writing the program for the ibex core, custom instructions can be used as described in Section 3.6.1. They cannot automatically be compiled. If the program is not very long and does not use many custom instructions, nops could be placed instead of the custom instructions and the program could be compiled like that. After the hexadecimal representation of the machine code has been obtained, the nops can manually be replaced by the hex value of the desired custom instructions.

The more elegant solution would be to modify the compiler, for instance GCC, to compile the custom instructions. This solution is preferred for longer programs, if many custom instructions are used, or simply if many programs are written to automate the compilation. This would be less error-prone and would speed up the process, because less manual intervention is needed.

| registers | usage | scope |
|---|---|---|
| s0 | **instr_page_size**: parameter written to SRAM by the mgmt core; size of the physical instruction pages; should be >= 4 and <= 32 | needed the entire time while the ibex core is running and getting new instruction pages; the callee needs to save this value on the stack and restore it before calling the paging function if it wants to use this reg (as the s regs are intended) |
| s1 | **instr_nr_pages**: parameter written to SRAM by the mgmt core; needs to be equivalent to 64 / instr_page_size | same as s0 |
| s2 | **instr_start_addr_next_page**: the address where the instruction page starts, so the core can jump there | same as s0 |
| s3 | **instr_page_counter**: logical page counter that gets incremented for every page | same as s0 |
| s4 | **instr_page_counter_modulo**: physical page counter; equal to instr_page_counter modulo instr_page_size | same as s0 |
| s5 | **addr_function_req_next**: constant = 0xac; the address of the function req_next in page C | same as s0 |
| s6 - s11 | free for use by programmer | free |
| a0 | **bitstr_page_size**: parameter written to SRAM by the mgmt core; size of the physical bitstream pages; should be >= 4 and <= 16 | only needed while bitstream is loaded in phase 2 |
| a1 | **bitstr_nr_pages**: parameter written to SRAM by the mgmt core; needs to be equivalent to 32 / bitstr_page_size | same as a0 |
| a2 | **bitstr_words**: total number of words in bitstream (full bitstream: 4506 words) | same as a0 |
| a3 - a7 | free for use by programmer | free |

Table 2: The registers used by the ibex program.

# 5 CRC application

To demonstrate how the ICESOC can be used, a simple CRC demo program is written. It makes use of the paging mechanism explained in Section 4 to load the instruction pages page-wise into the SRAM. Two versions are written, one software-only version that is executed only on the ibex core, as well as one accelerated version, which uses a CRC custom instruction on the eFPGA. CRC is picked as a demo application because it is relatively simple to implement and widely used for error detection, for example in Ethernet, Gzip, and PNG [19], and therefore has many possible use cases.

## 5.1 CRC-32

CRC stands for cyclic redundancy check and is a checksum algorithm to detect data inconsistency during transmission [20]. An input message is interpreted as coefficients of a polynomial, which is divided by a fixed generator polynomial. The resulting remainder of the division is the checksum. It is appended to the message before transmitting. The receiver of the message can verify that dividing the message, including the checksum, by the generator polynomial results in a remainder of zero. If that is not the case, the message has been corrupted.

CRC-32, sometimes called CRC-32B [21] is a CRC algorithm, that uses a 32-bit generator polynomial and therefore produces a 32-bit remainder, the checksum. Usually, the generator polynomial 0x104c11db7 is used. The leading one is implicit, so it can be represented as a 32-bit word. Depending on the exact implementation, the reflected version of this polynomial can be used, which is 0xedb88320 [21]. In the polynomial arithmetic used by CRC, addition and subtraction are equivalent to an XOR operation [20], which simplifies the computation. When implementing a CRC-32 calculation, the CRC state can be shifted left or right. Depending on the direction, either each byte in the input or the generator polynomial has to be flipped. The flipped generator polynomial is the reflected polynomial stated above. Both implementations are correct, but they need to be consistent, as explained in [21]. For the following, approach b, the Formulaic "Reflected" CRC32 from [21] was used as a reference. It uses a right shift and the reflected polynomial. However, it was adapted slightly to iterate over entire input words instead of bytes at a time. Since the custom instruction accepts two registers as source registers, using an in input size a multiple of the word size is a natural choice. This way an entire word can be processed at once on the fabric. That the assumption about the input size holds needs to be ensured manually when calling the function.

The CRC-32 checksum can be calculated by following the steps in Fig. 11.

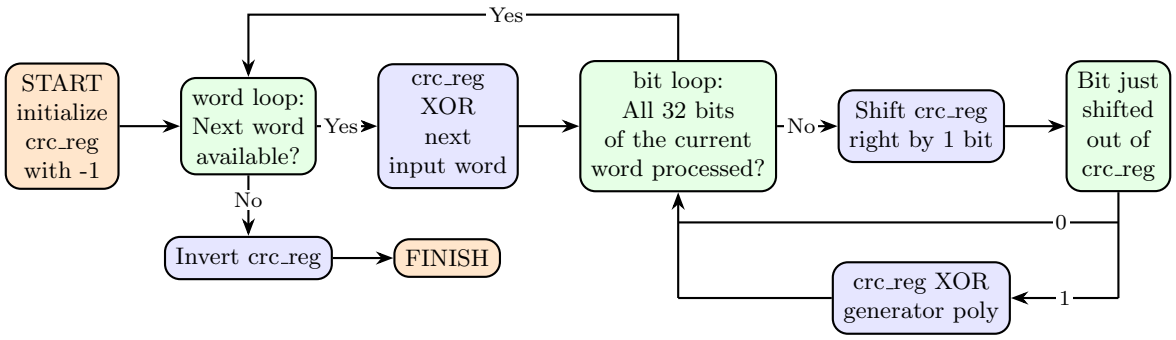


Figure 11: The CRC-32 algorithm. (AI was used to generate and iteratively modify this diagram.)

First the CRC state is initialized to -1, which corresponds to all ones in binary. Then the word loop is entered, which continues while a next input word is available, and terminates as soon as the

input has been processed completely. The current CRC state is XORed with the next input word that has just been read and the bit loop is entered. It loops over every bit in the current input word, and checks the current LSb value. In both cases, the CRC state is shifted one bit to the right. If the LSb was one, the state is XORed with the generator polynomial, otherwise this step is skipped. Once the bit loop has iterated over all 32 bits in the current word, the next iteration of the word loop starts. Once all input words have been processed, the word loop terminates. Finally the CRC state is inverted bit-wise. The result is the CRC-32 checksum of the input.

Since this iterates over every bit, it is relatively slow. To speed it up, entire bytes could be processed at once without a loop. This is called the table-driven implementation [22]. For this, the division results for every possible input byte are precomputed and stored in a table. This table therefore contains 256 words. Since it needs to be stored somewhere and the memory in the ICESOC is scarce, this would not be a good option here. Instead the eFPGA fabric will be used to implement a CRC checksum calculation of one word much faster than if it were completely executed in software.

## 5.2 The program for the ibex core

Below two implementations for CRC-32 can be seen side by side. Fig. 12a shows a software-only implementation, which will be executed exclusively on the ibex core. Fig. 12b shows an implementation using a custom instruction on the eFPGA that updates the CRC state for a new input word. Both implementations follow the steps for calculating the CRC checksum as shown in Fig. 11. Register t0 contains the current CRC state. t1 temporarily holds the next input word. t2 contains the bit counter for the inner bit loop. t3 temporarily stores the LSb of the current CRC state. t4 is the generator polynomial. a0 and a1 are the input parameters of the function. a0 is the input data pointer and a1 is the input length in bytes.

```
1  crc32b_sw:   addi t0, zero, -1
2               lui t4, 0xEDB88
3               addi t4, t4, 800
4  word_loop1: beq a1,  zero, done1
5               lw  t1, 0(a0)
6               xor t0, t0, t1
7               addi t2, zero, 32
8  bit_loop1:   andi t3, t0, 1
9               srli t0, t0, 1
10              beq t3, zero, no_xor1
11              xor t0, t0, t4
12 no_xor1:     addi t2, t2, -1
13              bne  t2,  zero, bit_loop1
14              addi a0, a0, 4
15              addi a1, a1, -4
16              jal zero, word_loop1
17 done1:       xori t0, t0, -1
18              add  a0, t0, zero
19              jalr zero, ra, 0
```

(a) The unaccelerated CRC-32B calculation (software version). (This was generated with AI and manually verified and modified.)

```
1  crc32b_acc: addi t0, zero, -1
2
3
4  word_loop2: beq  a1, zero, done2
5              lw   t1, 0(a0)
6              nop # eFPGA0d0 t0,t0,t1
7                  # -> 0x0062c28b
8
9
10
11
12
13
14             addi a0, a0, 4
15             addi a1, a1, -4
16             jal  zero, word_loop2
17 done2:      xori t0, t0, -1
18             add  a0, t0, zero
19             jalr zero, ra, 0
```

(b) The accelerated CRC-32B calculation. The nop in line 6 has to be replaced by the custom instruction call after compilation.

Figure 12: The software-only and accelerated CRC functions.

Equivalent lines are placed side by side, so it can be seen which instructions from the software version are replaced with just one call to the custom instruction in line 6 of Fig. 12b. The accelerated version contains a nop where the custom instruction will be placed. The nop can easily be swapped out manually by the hex value commented out in line 7 after obtaining the compiled version. This is easier than modifying the compiler if just one custom instruction call is needed.

By design both implementations are short enough to fit into an instruction page of 32 words, while leaving enough room in that page for the two obligatory paging function calls to instruction page C, as explained in Section 4, and the call to the CRC function itself. The latter could of course be omitted by just placing the code of the function where the function call would have been, but using a function call makes it more readable. Storing register values on the stack when entering the function is omitted due to constaints in the number of instructions in a page.

Both functions are called with the two parameters a0, and a1. a0 contains a pointer to the input data. a1 is the length in bytes of the input data. The length needs to be a multiple of four to ensure only complete words are processed. Those two parameters and the input data itself are written into SRAM by the mgmt core before the ibex core executes the page containing the CRC calculation. The ibex core initializes a0 and a1 with the values it reads from memory before calling the functions. Therefore the firmware of the management core provides the input data for the CRC calculation. The resulting checksum is written to memory by the ibex core, so it can be checked by reading the corresponding value from memory after the simulation has terminated.

The accelerated version of the CRC function contains 9 instructions less than the unaccelerated one. This means, 9 words less have to be stored in SRAM to implement the same functionality. The software version Fig. 12a has a word loop which iterates over an inner bit loop. This bit loop consists of five or six instructions, depending on the value of the current bit, which is the LSb of the current CRC value. If it is zero, the branch is taken and the XOR instruction is skipped. The loop then takes five instructions. If it is one, the XOR needs to be executed and the loop takes six instructions. So to process one word, the inner bit loop executes between $32 * 5 = 160$ and $32 * 6 = 192$ instructions. The outer word loop executes 167 to 199 instructions per iteration. In the accelerated version, the entire bit loop, and the XOR and addi instructions immediately before it are replaced by the custom instruction which will be executed on the eFPGA. That corresponds to lines 6 to 13 in Fig. 12a, which are replaced by only one custom instruction in line 6 in Fig. 12b. Also the loading of the generator polynomial into register t4 in lines 2 and 3 can be omitted in the accelerated version.

In summary for the entire CRC calculation the software-only version executes 7 + 167 * nr_input_words instructions in the best case and 7 + 199 * nr_input_words instructions in the worst case. The accelerated version only needs 5 + 6 * nr_input_words in any case. Therefore the accelerated version executes 161 to 193 instructions less per input word. The actual speedup this can provide depends on how many clock cycles each instruction takes. To obtain the speedup, a clock cycle counter can be used to measure how long both versions take. This is explained in Section 5.4.

## 5.3 The custom instruction for the eFPGA fabric

The design for the custom instruction on the eFPGA can be seen in Listing 6. Wopa is the current state of the CRC register, while wopb is the next word of the input. 33 bundles of wires are created, the stages. Every stage represents the result of one iteration of the bit loop in Fig. 11. For every stage, multiplexer controlled by the current LSb decide if the last stage is just shifted or also XORed with the generator polynomial. This entire calculation is done combinationally in one deep multiplexer tree. So all 32 stages are executed in one clock cycle. The result is stored in a register, which is only

```
1       reg [31:0] wres0_reg;
2       localparam [31:0] POLY = 32'hEDB88320;
3       wire [31:0] stage [0:32];
4       assign stage[0] = wopa ^ wopb;
5       genvar i;
6       generate
7           for (i = 0; i < 32; i = i + 1) begin : round // this generates a deep combinational
            ↪  MUX tree, which executes in one clock cycle in RTL simulation, but should be
            ↪  pipelined to not limit the max clk frequency
8               assign stage[i+1] = stage[i][0] ? ((stage[i] >> 1) ^ POLY)
9                                               : (stage[i] >> 1); // always shift right by one
                                                ↪  bit and conditionally XOR with the
                                                ↪  polynomial
10          end
11      endgenerate
12      always @(posedge clk) begin
13          wres0_reg <= stage[32]; // the result is assigned at the rising clock edge
14      end
15      assign wres0 = wres0_reg;
16      assign wres1 = 32'hffffffff;
17      assign wres2 = 32'hffffffff;
```

Listing 6: The CRC custom instruction on the eFPGA. An entire input word is processed in one clock cycle here. (This was generated with AI and manually verified and modified.)

updated with the clock. This prevents the output from toggling multiple times per clock period, which would lead to a very slow simulation. The fabric alone can be simulated this way, but as soon as the ICESOC is simulated with it, it gets too slow, because all those toggles on the fabric output could trigger changes in the ICESOC. In RTL simulation, this deep multiplexer tree is no problem, since the timing is not an issue. When running this design on the actual chip, the clock frequency needs to be taken into account. The report generated by Yosys and nextpnr when generating the bitstream is shown in Listing 9.

It claims that the design in Listing 6 can run at a maximum clock frequency of 30.40MHz. This constrains the clock of the entire chip to this maximum frequency. To circumvent this, the design could be pipelined by adding registers. This could be done as aggressively as needed to match timing requirements. Then a faster clock could be used, but the custom instruction would need multiple clock cycles to execute. In the program for the ibex core the delay field of the custom instruction would simply need to be adapted as shown in Fig. 5. With this, the ibex core will wait for the custom instruction to terminate. If the design uses more than 15 clock cycles, the eFPGA delay field can be set to 4'b1111, so the eFPGA done signal would be used. This is bit 34 of W_RES1 as shown in Fig. 4 and would need to be set as part of the implementation of the custom instruction. Currently only one custom instruction is used in slot 0. The results of the other two slots are set to all ones, which can be seen in lines 16 and 17 of Listing 6. This could easily be changed to implement different custom instructions as needed. The device utilization report in Listing 9 shows that the current design uses 192 of the 720 available LUTs on the fabric. This is less than one third, so enough resources should be available to implement custom instructions in the other slots. The I/O resources and the global clock are used 100%, which is not a problem since the clock and the I/Os of the fabric are all implemented here already and can be used by the future custom instructions in slot 1 and 2.

## 5.4 Comparing performance of the accelerated and the unaccelerated CRC

To compare the execution times of the software-only and the accelerated CRC functions, the number of clock cycles can be counted. Ibex supports the hardware performance counters extension Zihpm, which includes a clock cycle counter [23]. The value of this clock counter is stored in the two registers mcycle and mcycleh. Mcycle stores the 32 lower bits, while mcycleh keeps track of the upper 32 bits. Those registers can be read with the instruction

```
csrr rd, mcycle
```

To calculate how many clock cycles have passed while a function was executing, the state of the mcycle register is read before calling the function and immediately after it has returned. This can be seen in Listing 7.

```
1 csrr t5, mcycle                # read lower 32 bits of clock counter into t5
2 jal ra, crc32b_{sw or acc}     # CRC function call (unaccelerated or accelerated)
3 csrr t6, mcycle                # read lower 32 bits of clock counter into t6
4 sw a0, 0x10(zero)              # write crc result to mem[0x10]
5 sub t6, t6, t5                 # subtract nr cycles at start of crc from nr cycles at end
6 sw t6, 0x14(zero)              # write nr clk cycles to mem[0x14]
```

Listing 7: Measuring the number of clock cycles the CRC function needs.

Here, the value of mcycleh is ignored, since this example application should not run for such a long time that the mcycle register overflows into mcycleh. If this would happen, like when using a function that runs for much longer, the mcycleh register would also need to be considered. In Listing 7, in line 1, the current value of the mcycle register is read into t5, then the CRC function is called. After the function returned, the new value of mcycle is read into register t6 in line 3. Next, the result of the CRC computation is written to SRAM, so it can be evaluated later. The two mcycle values are subtracted in order to obtain the number of clock cycles passed since the invocation of the CRC function. This number of clock cycles is also stored to memory in line 6. That is done for both functions, the accelerated and unaccelerated versions, so in the end, their results and the number of clock cycles they needed can be compared.

Table 3 contains a comparison of the software only and the accelerated functions. In the following aspects, the accelerated version has an advantage over the software-only version. First of all, because nine instructions from the software-only program can be replaced by one custom instruction call for the accelerated version, the latter has a smaller program size and is more memory efficient. In systems with such a harsh memory constraint like the ICESOC, this is a relevant factor. Secondly, as explained in Section 5.2, the accelerated version executes 161 to 193 instructions less per input word than the software version. This leads to a speedup, which depends on the number of cycles each instruction takes to execute. The implementation of the custom instruction used here can process an entire 32 bit word in one clock cycle. This results in just 23 clock cycles needed for the CRC calculation of a 32 bit input. In comparison, the software-only approach takes 294 clock cycles. For an input size of four words, the computation was sped up from 1135 to just 56 clock cycles, which is a speedup of around 20.27x.

This single-cycle custom instruction works for the RTL simulation since meeting timing requirements is not a factor here. As stated in Listing 9, the max clock frequency for this design is 30.40MHz. When keeping the single-cycle design, the clock frequency of the entire chip is limited by this constraint. This is the first drawback of the accelerated version. To circumvent this, the design for the

custom instruction could be pipelined with as many stages as needed to meet timing requirements. Then a faster clock could still be used and the custom instruction would require multiple clock cycles. Since the accelerated version uses at least 161 instructions per word less, a speedup would be achieved even if the custom instruction used 32 pipeline stages and would need 32 clock cycles to execute. Furthermore, the accelerated program cannot run on just any RISC-V core, like the software-only version does. A hardware implementation of the custom instruction is needed that also matches the delay specified as part of the instruction. This requires either an eFPGA fabric like in the ICESOC, or a static hardware implementation of this functionality. This requirement of additional hardware aside from the RISC-V core running the main program also leads to a higher area needed by the accelerated version.

One advantage the accelerated version on an eFPGA has compared to using a static hardware accelerator is its reconfigurability. The custom instructions can be adapted to the program the RISC-V core is currently running. If the requirements on clock frequency would change or a different CRC algorithm should be used, this design can simply be updated and the fabric can be configured with the newly generated bitstream.

| | **unaccelerated CRC-32B (ibex core only)**<br><br>**crc32b_sw Fig. 12a** | **accelerated CRC-32B (custom instruction on the eFPGA fabric)**<br>**crc32b_acc Fig. 12b** |
|---|---|---|
| **# instructions executed** | worst case:<br>7 + 199 * nr_input_words<br>best case:<br>7 + 167 * nr_input_words | 5 + 6 * nr_input_words |
| **program size** | 19 instructions | 10 instructions<br>needs 47.37% less instruction memory |
| **# clock cycles** | 294 for a one word input<br>1135 for a four word input<br>(1135-294)/3=280 cycles per word (without initial overhead, exact value depends on the input data) | 23 for a one word input (12.78x speedup)<br>56 for a four word input (20.27x speedup)<br>(56-23)/3=11 cycles per word (without initial overhead)<br>converges to 280/11=25.45x speedup |
| **portability** | can run on any other RISC-V core | needs the custom instruction to be supported by HW (like the eFPGA fabric) |
| **area** | requires just the RISC-V core | needs HW support like an eFPGA fabric in addition to the RISC-V core |
| **max clk frequency** | no additional constraints on max clk frequency | limited by the implementation of the custom instruction<br>this design: max clk frequency: 30.40 MHz (design could be pipelined to use a faster clock) |

Table 3: Comparing the software-only implementation for the ibex core (unaccelerated version) Fig. 12a with the accelerated version using a custom instruction on the eFPGA Fig. 12b.

In conclusion, the speedup of the accelerated CRC calculation converges to about 25x for large input data. Every additional word takes 11 cycles in the accelerated version compared to around 280 cyles in the software-only version. The exact number of clock cycles in the unaccelerated version depends on the value of the input data and how many times the branch in the bit loop is taken, so this is just a rough estimate. Furthermore the accelerated version needs 47.37% less instruction memory for just the CRC function. This can be helpful in navigating the memory constraints of the ICESOC.

Accelerating a program by using a custom instruction on an eFPGA can be helpful when the part of the program that is being accelerated is used often and can be efficiently implemented on the fabric. The more instructions can be replaced, the higher the speedup. For this CRC application, this worked well, because an entire loop is replaced, which iterates over 32 bits. On the fabric this is done in one deep combinational MUX tree in only one clock cycle. This constrains the maximum clock frequency, but the design could be pipelined to loosen this constraint.

It is important to note that this is not the only way of accelerating a computation and might not be useful for every application. Since the software depends on the availability of the fabric, this does not work if different hardware is used. As stated in [24], different techniques can be used to increase the performance of a CPU. There, a reconfigurable soft CPU was investigated, but many of the ideas still apply here. If the custom instructions on the fabric are dynamically reconfigured, which is not done here but possible in theory, the overhead of configuration needs to be considered. Since it takes long to configure the eFPGA, the speedup is only preserved if the custom instruction is used often enough that its accumulated speedup outweighs the slowdown through configuration. If it rarely used, it might be better to emulate the instruction in software. A custom instruction that is used often throughout the entire program should stay on the fabric, while instructions that are used in bursts can be swapped out when they are not used. Another option could be to optimize the software. This also offers a big potential for speedup. As an example, the software could be modified to use as little instructions as possible by using a more efficient algorithm or to reduce memory accesses by keeping certain data in the registers instead of loading it multiple times. This only works if the data does not change between accesses and if enough unused registers are available.

The CRC-32 calculation specifically can be accelerated by using a table-driven approach, as introduced in Section 5.1. Since this needs a table of 256 32-bit values stored in memory, it is not suitable here, because not enough SRAM is available in the user project. In other systems, this might be the preferred approach.

# 6 Conclusion

An eFPGA fabric providing hardware support for custom instructions can be used to accelerate a RISC-V core. The ICESOC aims to fulfill this purpose by tightly coupling an eFPGA fabric with two ibex RISC-V cores. The fabric has three slots in which it can contain up to three custom instructions that can be called by the cores. They pass two operands directly from the register file to the fabric. The result from the selected slot is written back to the register file as well. Configuring the eFPGA can be done either through UART, bitbang, or one of the cores. When the core is executing a specific custom instruction, one word of the configuration bitstream is written from its register to the fabric. The instructions that the core will execute as well as the bitstream to configure the fabric will be read from SRAM by the core. Writing this data to SRAM could be done through UART. The issue here is, that not all data can be stored in SRAM at the same time, so the core will have to communicate with the entity which is providing the data to ensure, it is provided when needed. However, the UART output of the entire ICESOC is disabled because its low-active output enable signals are flipped. Therefore the management core inside the Caravel harness has to be used to write the instruction and bitstream data to SRAM. When addressing the SRAM, there is a mismatch in the address signals of the core and the SRAM. The core's byte-address is connected to the SRAMs word-address. For this reason, only every fourth word in SRAM is addressable by the cores. The two SRAMs in the user project of the ICESOC have a size of 1KB each. Because only a quarter can be used, and the word size is 32 bit, each SRAM can store only 64 words. To solve this, an SRAM paging mechanism was implemented here, which splits the SRAM up into pages, that are provided by the management core as requested by the ibex core. To demonstrate the use of the SRAM paging and to prove that a speedup can be achieved by using the custom instructions on the fabric to accelerate the cores, an example CRC application was implemented. It showed a speedup of around 20x for a four word input. A 25x speedup is expected for larger inputs. To get this result, a software-only implementation using only the instructions in the RISC-V core was written first. Then an entire program loop, which iterates 32 times for every input word, was replaced. Instead, a single-cycle custom instruction implemented on the eFPGA was called. This is the accelerated implementation, whose clock cycle count was compared to the one of the software-only implementation to obtain the speedup.

Using custom instructions on an eFPGA fabric to accelerate a computation is not a universal solution. However it can be beneficial, when a program can be shortened by several instructions because they can be encapsulated in one custom instruction that can be implemented efficiently in hardware. If that custom instruction executes faster on the fabric than the equivalent sequence of RISC-V instructions needed for a software-only solution, this results in a speedup. The more often the custom instruction is used, the higher the speedup.

Future work could include dynamic reconfiguration of the fabric during program execution. If the fabric is only configured once at the beginning, as it is now, the initial configuration overhead will be compensated eventually, since every call to the custom instruction provides a speedup. Reconfiguring introduces an overhead every time. This needs to be taken into account when evaluating the possible benefits of reconfiguration. Another possibility would be to use the second core in addition to the first one. Here the small SRAM would be a challenge if both cores need distinct instruction and data, so the tradeoffs need to be carefully considered.

# Glossary

**ALU** Arithmetic logic unit. 13, 37

**ASCII** American Standard Code for Information Interchange. 37

**Caravel harness** A "test harness for creating designs with the Google/Skywater 130nm Open PDK" [25]. The source code can be found at [15]. 5, 6, 15, 37

**CISC** Complex instruction set computer. 2, 37

**CPI** Clock cycles per instruction. 2, 37

**CPU** Central processing unit. 3–5, 35, 37

**CRC** Cyclic redundancy check. , 1, 26, 29–37

**CRC-32** Cyclic redundancy check, version using a 32-bit polynomial. 29, 30, 35, 37

**DISC** Dynamic instruction set computer [12]. 4, 37

**EDA** Electronic design automation. 4, 37

**eFPGA** Embedded field programmable gate array. , 1, 3–6, 8, 11–13, 17, 20, 21, 23, 26, 29–32, 34–37

**FIFO** First In - First Out. 18, 37

**FlexBex** Predecessor of the ICESOC, introduced in [1]. 3, 37

**FPGA** Field programmable gate array. 4, 37

**FSM** Finite-state machine. 13, 14, 37

**GARP** A MIPS processor with an FPGA as reconfigurable coprocessor [11]. 4, 37

**GCC** GNU Compiler Collection, formerly GNU C Compiler. 25, 27, 37

**GPIO** General-purpose input/output. 6, 11, 37

**HW** Hardware. 34, 37

**I/O** Input/output. 32, 37

**ibex core** An open source 32 bit RISC-V CPU core [17]. 1, 4, 5, 7–13, 17–32, 37

**ICESOC** Multicore SoC with two RISC-V cores, two 1KB SRAMs, and a T-shaped eFPGA fabric to host custom instructions [14]. , 1, 3–6, 10, 11, 17, 18, 26, 29, 30, 32–37

**ISA** Instruction set architecture. , 1–3, 8, 37

**LSb** Least significant bit. 9, 10, 12, 30, 31, 37

**LUT** Lookup table. 32, 37

**mgmt core** The Caravel management core. 9, 15, 19–23, 25, 26, 28, 31, 37

**MIPS** Microprocessor without Interlocked Pipeline Stages. A family of RISC ISAs. 4, 37

**mprj** Mega project. The user project inside the Caravel harness.. 5, 37

**MPW** Multi-project wafer. 5, 37

**MSb** Most significant bit. 7, 9, 10, 37

**mutex** MUTual EXclusion. Shared variable to control access to a shared resource. 17, 18, 37

**MUX** Multiplexer. 35, 37

**nop** No operation instruction. 25, 27, 30, 31, 37

**PDK** Process design kit. 4, 5, 37

**RISC** Reduced instruction set computer. 2, 3, 37

**RISC-V** An open source ISA that follows the RISC philosophy. , 1, 3–6, 8, 17, 34, 36, 37

**RTL** Register-transfer level. 1, 32, 33, 37

**RV32I** The RISC-V 32 bit base integer instruction set with full 32 registers (as compared to RV32E with only 16 registers). 8, 37

**SoC** System on a chip. , 1, 4–6, 37

**sp** Stack pointer. 24, 37

**SRAM** Static random-access memory. , 1, 4–10, 15, 17–29, 31, 33, 35–37

**UART** Universal asynchronous receiver-transmitter. , 1, 6–11, 15, 36, 37

**XOR** Exclusive or. 29–31, 37

# References


[1] Nguyen Dao et al. "FlexBex: A RISC-V with a Reconfigurable Instruction Extension". In: *2020 International Conference on Field-Programmable Technology (ICFPT)*. 2020 International Conference on Field-Programmable Technology (ICFPT). Dec. 2020, pp. 190–195. DOI: 10.1109/ICFPT51103.2020.00034. URL: https://ieeexplore.ieee.org/document/9415607 (visited on 07/07/2026).

[2] David Patterson. "Reduced Instruction Set Computers Then and Now". In: *Computer* 50.12 (Dec. 2017), pp. 10–12. ISSN: 1558-0814. DOI: 10.1109/MC.2017.4451206. URL: https://ieeexplore.ieee.org/document/8220478 (visited on 07/07/2026).

[3] John L. Hennessy, David A. Patterson, and Krste Asanović. *Computer architecture: a quantitative approach*. 5th ed. Waltham, MA: Morgan Kaufmann/Elsevier, 2012. 493 pp. ISBN: 978-0-12-383872-8.

[4] Soham Chatterjee. *A Beginner's Guide to RISC and CISC Architectures*. Medium. Oct. 14, 2018. URL: https://medium.com/@csoham358/a-beginners-guide-to-risc-and-cisc-architectures-fc9af424db3b (visited on 07/08/2026).

[5] *Never heard of microcode? Nope, us neither. But it's really important.* DEV Community. Jan. 30, 2026. URL: https://dev.to/dimension-ai/never-heard-of-microcode-nope-us-neither-but-its-really-important-14j8 (visited on 07/08/2026).

[6] Khushi Jain. *RISC vs CISC — The Architecture Battle That Shaped Modern Computing*. Medium. Mar. 15, 2026. URL: https://medium.com/@khushi.jain24/risc-vs-cisc-the-architecture-battle-that-shaped-modern-computing-efde7a3811cc (visited on 07/08/2026).

[7] *RISC-V Ratified Specifications Library :: RISC-V Ratified Specifications Library*. URL: https://docs.riscv.org/reference/home/index.html (visited on 07/20/2026).

[8] Biagio Peccerillo et al. "A survey on hardware accelerators: Taxonomy, trends, challenges, and perspectives". In: *Journal of Systems Architecture* 129 (Aug. 1, 2022), p. 102561. ISSN: 1383-7621. DOI: 10.1016/j.sysarc.2022.102561. URL: https://www.sciencedirect.com/science/article/pii/S1383762122001138 (visited on 07/09/2026).

[9] C. Cascaval et al. "A taxonomy of accelerator architectures and their programming models". In: *IBM Journal of Research and Development* 54.5 (Sept. 2010), 5:1–5:10. ISSN: 0018-8646. DOI: 10.1147/JRD.2010.2059721. URL: https://ieeexplore.ieee.org/document/5571946 (visited on 07/21/2026).

[10] Bea Healy et al. "Virtualization and Dynamic Reconfiguration of Custom Instruction Accelerators (CIA) in RISC-V Embedded Systems". In: *2025 35th International Conference on Field-Programmable Logic and Applications (FPL)*. 2025 35th International Conference on Field-Programmable Logic and Applications (FPL). ISSN: 1946-1488. Sept. 2025, pp. 1–9. DOI: 10.1109/FPL68686.2025.00013. URL: https://ieeexplore.ieee.org/document/11449097 (visited on 07/20/2026).

[11] J.R. Hauser and J. Wawrzynek. "Garp: a MIPS processor with a reconfigurable coprocessor". In: *Proceedings. The 5th Annual IEEE Symposium on Field-Programmable Custom Computing Machines Cat. No.97TB100186)*. . The 5th Annual IEEE Symposium on Field-Programmable Custom Computing Machines Cat. No.97TB100186). Apr. 1997, pp. 12–21. DOI: 10.1109/FPGA.1997.624600. URL: https://ieeexplore.ieee.org/document/624600 (visited on 07/07/2026).

[12] M. J. Wirthlin. "A dynamic instruction set computer". In: *Proceedings of the IEEE Symposium on FPGA's for Custom Computing Machines*. FCCM '95. USA: IEEE Computer Society, Apr. 19, 1995, p. 99. ISBN: 978-0-8186-7086-2. (Visited on 07/07/2026).

[13] Leo Moser et al. "Greyhound: A Reconfigurable and Extensible RISC-V SoC and eFPGA on IHP SG13G2". In: *2025 Austrochip Workshop on Microelectronics (Austrochip)*. 2025 Austrochip Workshop on Microelectronics (Austrochip). ISSN: 2689-8144. Sept. 2025, pp. 5–8. DOI: 10.1109/Austrochip67945.2025.11183709. URL: https://ieeexplore.ieee.org/document/11183709 (visited on 07/21/2026).

[14] nguyendao-uom. *nguyendao-uom/ICESOC*. original-date: 2021-12-28T02:38:07Z. Mar. 27, 2026. URL: https://github.com/nguyendao-uom/ICESOC (visited on 07/07/2026).

[15] *efabless/caravel*. original-date: 2021-10-07T18:32:22Z. July 1, 2026. URL: https://github.com/efabless/caravel (visited on 07/07/2026).

[16] *Chip Gallery*. FABulous: Open-Source Embedded FPGA Framework. URL: https://fabulous.readthedocs.io/en/latest/gallery/index.html (visited on 07/20/2026).

[17] *Ibex: An embedded 32 bit RISC-V CPU core — Ibex Documentation 0.1.dev50+g8ed87e07e documentation*. URL: https://ibex-core.readthedocs.io/en/latest/ (visited on 07/20/2026).

[18] Andrew S. Tanenbaum and Herbert Bos. *Modern operating systems*. 4. ed. Boston: Prentice Hall, 2015. 1101 pp. ISBN: 978-0-13-359162-0 978-1-292-06142-9.

[19] Mark Benly. *Understanding the CRC32 Header: A Deep Dive into Cyclic Redundancy Check*. Medium. Feb. 14, 2026. URL: https://medium.com/@mark.benly/understanding-the-crc32-header-a-deep-dive-into-cyclic-redundancy-check-3b34d31585c7 (visited on 08/12/2026).

[20] *Sunshine's Homepage - Understanding CRC*. URL: https://www.sunshine2k.de/articles/coding/crc/understanding_crc.html (visited on 08/12/2026).

[21] Michael "Code Poet" Pohoreski. *Michaelangel007/crc32*. original-date: 2017-02-01T16:50:00Z. Aug. 8, 2026. URL: https://github.com/Michaelangel007/crc32 (visited on 08/09/2026).

[22] *ross.net/crc/download/crc_v3.txt*. URL: http://ross.net/crc/download/crc_v3.txt (visited on 08/11/2026).

[23] *Performance Counters — Ibex Documentation 0.1.dev50+g3250d9948 documentation*. URL: https://ibex-core.readthedocs.io/en/latest/03_reference/performance_counters.html (visited on 08/12/2026).

[24] Alexander Wold, Dirk Koch, and Jim Torresen. "Design techniques for increasing performance and resource utilization of reconfigurable soft CPUs". In: *2012 IEEE 15th International Symposium on Design and Diagnostics of Electronic Circuits & Systems (DDECS)*. 2012 IEEE 15th International Symposium on Design and Diagnostics of Electronic Circuits & Systems (DDECS). Apr. 2012, pp. 50–55. DOI: 10.1109/DDECS.2012.6219024. URL: https://ieeexplore.ieee.org/abstract/document/6219024 (visited on 08/13/2026).

[25] *Efabless Caravel "harness" SoC — Caravel Harness documentation*. URL: https://caravel-harness.readthedocs.io/en/latest/ (visited on 07/07/2026).

# Appendix A

Verwendete KI-Hilfsmittel:

1. Google Gemini
2. ChatGPT
3. Claude Code

Ziele, für die die KI-basierten Hilfsmittel in der vorliegenden Arbeit eingesetzt wurden:

1. Literaturrecherche
2. Anpassen von Makefiles und Generieren von Python Skripten
3. Einen Überblick über die existierende Codebase bekommen
4. Syntaxhilfe für LaTeX, Python, Verilog, Makefiles und Unterstützung beim Finden und Beheben von Fehlern
5. Generieren von Assembly-Programm und top-Design für die CRC Anwendung

Verwendungsweise der KI-basierten Hilfsmittel:

1. Es wurde nach Quellen zu bestimmten Themen gefragt. Die vorgeschlagenen pQuellen wurden nach relevanten Passagen durchsucht und ggf. verwendet.
2. Python Skripte und Teile von Makefiles wurden generiert.
3. Um einen Überblick über die existierende und offen auf GitHub verfügbare Codebase zu bekommen wurde Gemini zu dem Code befragt. Manche Teile des Codes wurde Gemini gebeten zu erklären, um die Funktionsweise schneller zu erfassen. Dies hat das manuelle Studieren der Dateien nur ergänzt und nicht ersetzt.
4. KI wurde verwendet um bei der Suche nach Fehlern und deren Beheben zu helfen.
5. Claude Code und Gemini wurden verwendet um den Assembly Code und die custom instruction für die CRC Anwendung zu generieren. Die generierten Passagen wurden teils manuell überarbeitet und ihr Ergebnis mit einem online CRC Rechner für einige Beispiel inputs verifiziert.

Kapitel und Abschnitte der vorliegenden Arbeit, in denen die KI-basierten Hilfsmittel eingesetzt wurden, um Inhalte zu erzeugen:

1. In Kapitel Section 2 wurde KI als Unterstützung bei der Literaturrecherche verwendet und um einzelne Konzepte tiefer zu verstehen.
2. In Kapitel Section 3 wurde KI hautpsächlich verwendet um einen Überblick über den source code für den ICESOC zu bekommen.
3. In Kapitel Section 4 und Section 5 wurde KI verwendet um bei der Implementierung zu helfen. In Section 4 wurde KI nur unterstützend verwendet, vor allem um Makefiles und Python Skripte anzupassen oder Fehler zu finden. In Section 5 wurden das Programm für die Ibex core und das Design der custom instruction mit KI generiert und manuell angepasst und verifiziert.
4. Fig. 11 wurde mit Hilfe von Gemini erstellt und iterativ angepasst.

# Appendix B

```
module arbiter #(
parameter NUM_PORTS=5 //in the inter module, the arbiter is generated once for every of the three slaves with NUM_PORTS set to the number of masters, which is four
    )(
        clk,
        rst,
        request,
        grant
    );
    ...
        input clk;
        input rst;
        input [NUM_PORTS - 1:0] request;
        output reg [NUM_PORTS - 1:0] grant;0
        localparam WRAP_LENGTH = 2 * NUM_PORTS;
        ...
        integer yy;
        wire next;
        wire [NUM_PORTS - 1:0] order;
        reg [NUM_PORTS - 1:0] token;
        wire [NUM_PORTS - 1:0] token_lookahead [NUM_PORTS - 1:0];
        wire [WRAP_LENGTH - 1:0] token_wrap;
        assign token_wrap = {token, token};
        assign next = ~|(token & request); //if the current token holder is also requesting, next is set to 0, otherwise to 1
        always @(posedge clk) grant <= token & request; //if the current token holder is also requesting, its request is granted

        always @(posedge clk)
                if (rst)
                        token <= 'b1;
                else if (next)
                        for (yy = 0; yy < NUM_PORTS; yy = yy + 1) begin : TOKEN_
                                if (order[yy])
                                        token <= token_lookahead[yy];
                        end
        genvar xx;
        generate
                for (xx = 0; xx < NUM_PORTS; xx = xx + 1) begin : ORDER_
                        assign token_lookahead[xx] = token_wrap[xx+:NUM_PORTS];
                        assign order[xx] = |(token_lookahead[xx] & request);
                end
        endgenerate
endmodule
```

Listing 8: The round-robin arbiter used in the inter module. source: [14]

```
3.39. Printing statistics.
=== top_wrapper ===
        +----------Local Count, excluding submodules.
        |
    216 wires
    2109 wire bits
    57 public wires
    1950 public wire bits
    387 cells
        1   $scopeinfo
        1   Global_Clock
    10   IO_1_bidirectional_frame_config_pass
    56   InPass4_frame_config
    68   LUT2
    44   LUT3
    78   LUT4
    40   LUTFF
    89   OutPass4_frame_config
...
Info: Device utilisation:
Info:            FABULOUS_LC:     192/    720    26%
Info:   IO_1_bidirectional_frame_config_pass:      10/     10   100%
Info:   InPass4_frame_config:      56/     56   100%
Info:   OutPass4_frame_config:      89/     89   100%
Info:           RegFile_32x4:       0/     10     0%
Info:                 MULADD:       0/     14     0%
Info:           Global_Clock:       1/      1   100%
Info:          FABULOUS_MUX2:       0/    360     0%
Info:          FABULOUS_MUX4:       0/    180     0%
Info:          FABULOUS_MUX8:       0/     90     0%
Info:          Config_access:       0/     15     0%
...
Info: Max frequency for clock 'clk': 30.40 MHz (PASS at 12.00 MHz)
```

Listing 9: Excerpt from the Yosys and nextpnr output when generating the bitstream for the CRC32 application.